

Dangerzone

Take potentially dangerous PDFs, office documents, or images and convert them to a safe PDF.

Version 0.11.1 — 2026-10-22
https://docs.dangerzone.rocks/

Contents

Home 4

Get started 6

When should I use Dangerzone? 7
Convert your first document 9
Convert documents from the command line 12
Run dangerzone from source 15

Guides 18

Install Dangerzone 19
Verifying PGP signatures 25
Update Dangerzone 27
Update the sandbox 29
Using Podman Desktop 32
Report an issue 33
Development environment 36
Dangerzone Testing 45
Containerized environments 46
Debug the sandbox 48
Overview 50
Pre-release 51
Prepare build environments 53
Build release artifacts 55
QA 59
Release 63
Release notes templates 64
Managing Python installations 65
Using the Doit Automation Tool 67

Reference 68

Supported document formats 69
Operating System support 70
dangerzone-cli 71
dangerzone-image 74
dangerzone-machine 77
Sandbox protocol 80
Settings 83
Environment variables 85
Directories 86
Signing keys 88
Security policy 89
Overview 91
2024-12-24 92
2023-12-07 93
2023-10-25 94

Changelog 95
Third-party notice 110
Roadmap 112

Explanation 113
How Dangerzone works 114
Our Security model 116
Independent sandbox updates 119
Update notifications 121
Reproducible builds 126
Qubes OS integration 128
Create Dangerzone environments 130
The Dangerzone repositories 133
FAQ 136

What is Dangerzone, and why should I use it?

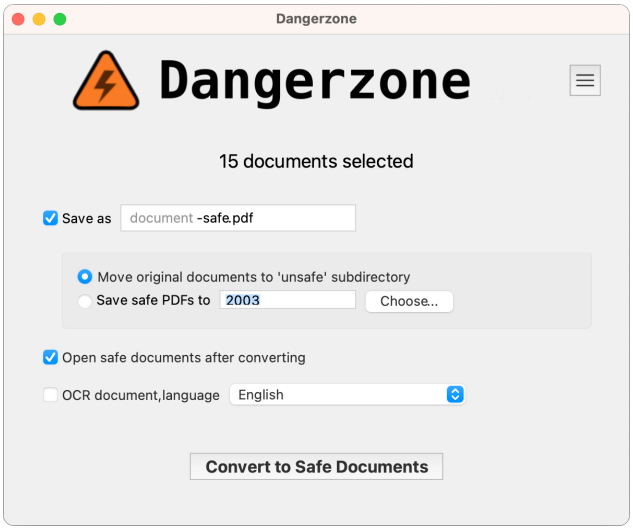
Dangerzone is a desktop application that helps you sanitize untrusted documents into safe PDFs. It supports more than 20 file types, including PDFs, all major office-suite formats, and the most common image types.

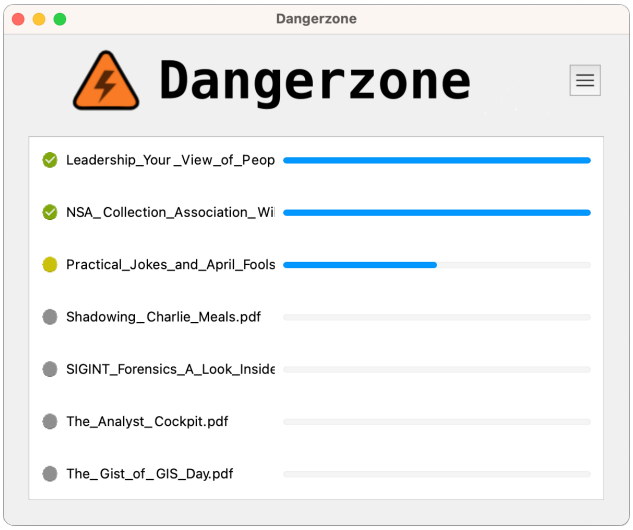
This is especially useful for journalists and activists, who may be targeted by law enforcement and bad actors.

It is a free and open source project, maintained by Freedom of the Press Foundation (FPF), a nonprofit organization that protects and defends press freedom.

You can read more about "How Dangerzone works", but the quick summary is this:

1. You give it a document you don't know if you can trust (for example, an email attachment)
2. Inside of a sandbox, it converts it to raw pixel data (a huge list of RGB color values for each page)
3. Then, outside of the sandbox, it reassembles this pixel data and generates a PDF from it





Features

- Sandboxes don't have network access, so if a malicious document can compromise one, it can't phone home
- Dangerzone can optionally do character recognition on the safe PDFs it creates, so they are searchable
- Dangerzone can convert many types of documents into safe PDFs.

> **Download this documentation for offline use**

If you prefer, you can also download this documentation as a PDF.

This documentation is organized in the following sections. Pick the one that matches what you are trying to do.

Tutorials (start here if you are new)

Learn how Dangerzone works and:

- Convert your first document
- Convert documents from the command line
- Run Dangerzone from source

How-to guides

Step by step guides to install on your platform, do more technical things like verifying signatures, updating the sandbox, building packages… or cutting a release.

→ How-to guides

Reference

Descriptions of the command-line tools, settings, environment variables, supported platforms and formats, as well as our security policy.

→ Reference

Explanation

What Dangerzone is and when to use it, a FAQ, and the design behind the sandbox and the security model.

→ Explanation

Get help

- Read the FAQ.
- Report bugs and ask questions on the issue tracker, following Report an issue.
- Want to help? Pick up an issue labelled good first issue and set up a development environment.
- Report security issues, following the security policy.
- Follow release announcements on Mastodon and on the official site.

See also

- GIJN Toolbox: Cutting-Edge — and Free — Online Investigative Tools You Can Try Right Now
- When security matters: working with Qubes OS at the Guardian

Dangerzone is developed by the Freedom of the Press Foundation and licensed under the AGPLv3. See the third-party notice for the software it depends on.

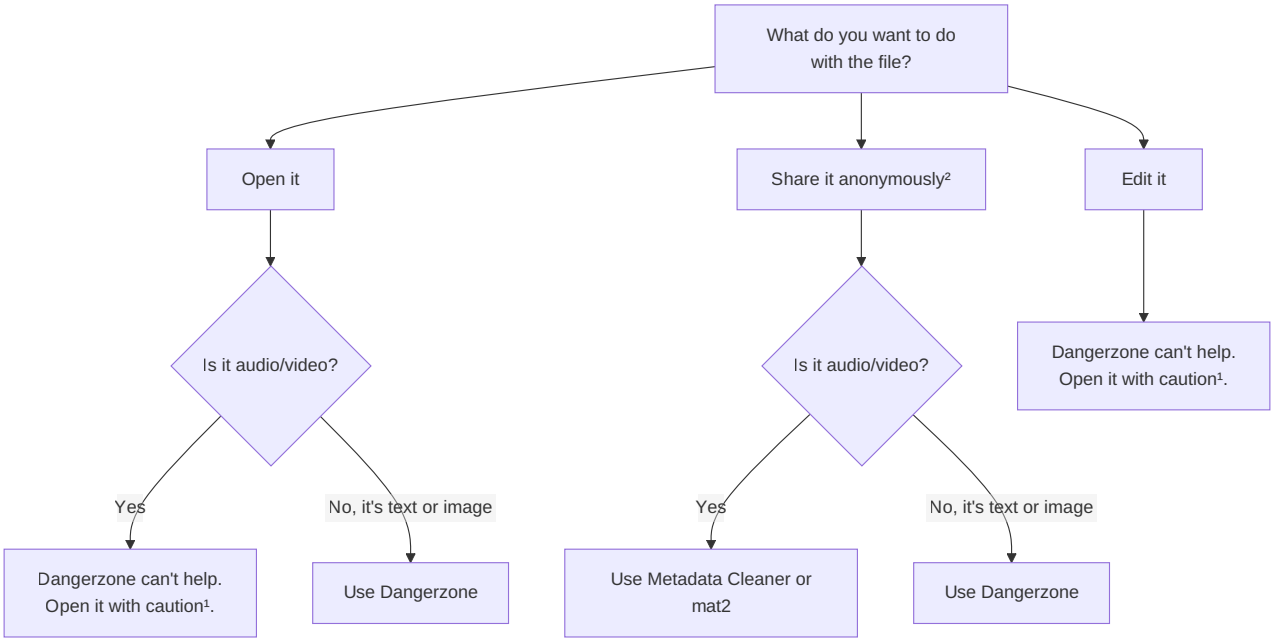
Get started

When should I use Dangerzone?

Dangerzone is the right tool for some situations and the wrong tool for others. The short rule:

> **Rule of thumb**
>
> Always use Dangerzone on text or image files you received from an external source. The exceptions are video or audio files (they are not supported at the moment), or documents you need to edit, as passing them to Dangerzone will transform them to PDF.

Decision guide



¹ Using Tails, or an equivalent system like Qubes is strongly recommended when handling untrusted files. ² Removing de-anonymization vectors from files is surprisingly difficult, and sometimes downright impossible, so proceed with **extra caution**.

Use Dangerzone when...

- **You received a document or an image and want to open it.** This is the main use case: an email attachment, a file from a tip line, a download from a site you don't fully trust. Dangerzone will render any malware or tracking in there ineffective.
- **You want to remove the metadata from a document.** The safe PDF carries none of the original metadata: no author name, no editing history, no camera model, no GPS position, nothing. Actually, if the document contained comments that weren't rendered, they will disappear as well.
- **You move a document from an untrusted environment to a trusted one.** For example from a machine used to browse the web to a machine that holds sensitive material. Converting on the way in keeps the trusted side clean.

Don't use Dangerzone when...

- **You need to keep editing the document.** Dangerzone rasterizes the file: the result is a picture of each page. This will remove formulas in a spreadsheet or macros in a Word document (that's actually the whole point!). If you must edit a document you don't trust, your best chance is probably to do it in a disposable environment instead.
- **The file is video or audio.** Dangerzone only handles documents and images (see supported formats). For removing metadata from media files, use Metadata Cleaner or mat2. For opening untrusted media, there is no Dangerzone equivalent. Your best chance is probably to do it in a disposable environment instead.
- **You want to know whether a file is malicious.** Dangerzone can't tell you if a file is safe. It just creates a copy that is. (Sorry, malware detection is another story)
- **You need to protect a source from fingerprinting of the document's content.** Dangerzone removes metadata and the file's structure, but doesn't change what is visible on the page: printer tracking dots, watermarks, unique wording, or steganography all survive because they are part of the picture. See the FAQ.

Examples

Situation	What to do
A source emails you a PDF	Use Dangerzone, then read the safe PDF
A colleague sends a spreadsheet you need to work on	Don't use Dangerzone. Open it in a disposable environment if you don't trust it
You photographed a document and want to publish it	Use Dangerzone to drop the camera metadata, after checking the picture itself for identifying details
Someone sends you a voice recording	Dangerzone can't help. Use Metadata Cleaner if you only need to strip metadata before sharing it
You downloaded a scanned form to print and sign	Use Dangerzone, then print the safe PDF

Convert your first document

In this tutorial you will install Dangerzone, open a document you don't fully trust, and turn it into a safe PDF. By the end, you will know what the application looks like, where the safe file ends up, and what happens to the original.

You need a computer running macOS, Windows, or a supported Linux distribution, an internet connection for the installation, and a document to convert. Any PDF, office document, or image works. Dangerzone retains the original and processes it without network connection. If you have nothing at hand, a PDF you downloaded from the web is a fine candidate.

1. Install Dangerzone

Follow the installation guide for your platform, then come back here:

- macOS (download the `.dmg`, or `brew install --cask dangerzone` if you use Homebrew)
- Windows (download the `.msi`, or `winget install FreedomofthePressFoundation.Dangerzone`)
- Ubuntu and Debian
- Fedora

Qubes OS and Tails users can follow along as well, using the Qubes OS and Tails guides.

2. Start Dangerzone for the first time

Launch Dangerzone from your applications menu.

The first time it starts, Dangerzone needs a *sandbox*: that's where the actual documents will be opened, in a secure way. What you see depends on how you installed it:

- On macOS and Windows, and with the `dangerzone-full` Linux packages, the sandbox is bundled. Dangerzone installs it and moves on.
- With the slim `dangerzone` Linux packages, a dialog titled **Download conversion sandbox?** asks whether Dangerzone may download the sandbox from the internet. Click **Yes, download sandbox and enable updates**. The download is a few hundred megabytes.

If the sandbox was bundled, Dangerzone asks the same question on the second start instead, with a dialog titled **Enable automatic sandbox updates?**. Click **Yes, enable sandbox updates** so that Dangerzone keeps the sandbox up to date and tells you when a new Dangerzone release is out. You can change your mind later from the hamburger menu in the top-right corner. See update notifications for what this setting does and does not do.

The main window then shows a large **Select suspicious documents ...** button. On the first run, a status bar at the bottom of the window shows the startup tasks that Dangerzone is performing, mainly starting a VM on Windows/macOS systems and installing the container image. You can proceed with selecting a document while the startup tasks run in the background.

3. Choose the document

Click **Select suspicious documents ...** and pick your document. You can also drag and drop one or more files onto the window.

The window switches to the settings view. It shows how many documents you selected and a **Change Selection** button if you picked the wrong one.

4. Review the settings

Leave the defaults for this first conversion, and simply read what they do:

- **Save as** shows the name of the output. The safe file gets the same name as the original with a `-safe.pdf` suffix, so `report.docx` becomes `report-safe.pdf`.
- **Move original documents to 'unsafe' subdirectory** is selected. After the conversion, the original is moved into an `unsafe/` folder next to the safe PDF, so that you don't open it by accident later.
- **Save safe PDFs to** lets you pick another output folder instead.
- **Open safe documents after converting** opens the result in your PDF viewer once it is ready. On Linux you can pick which viewer to use.
- **OCR document language** is checked, with English selected. OCR (optical character recognition) reads the text in the page images and adds an invisible text layer to the safe PDF, so you can still search and copy text. It works best when it knows the language, so pick your document's language from the list if it isn't English.

5. Convert

Click **Convert to Safe Document**.

Dangerzone opens the document inside the sandbox, renders every page to raw pixels, then rebuilds a PDF from those pixels outside the sandbox. A progress bar shows each stage. A one-page PDF takes a few seconds. A long office document with OCR takes longer.

When it is done, the document row shows a green check mark, and your PDF viewer opens the safe PDF if you left that setting on.

6. Look at the result

Open the folder that contains the original document. You will find:

- `<name>-safe.pdf`: the safe PDF. This is the file you can share, print, or keep.
- `unsafe/<name>`: the original, moved out of the way.

Open the safe PDF and check that all pages are there and that you can select text. Because the PDF was rebuilt from pixels, links, forms, and embedded scripts from the original are gone. That is the point.

What you learned

You have installed Dangerzone, let it fetch its sandbox, and converted a document while keeping the original archived. Every later conversion follows the same three steps: select, review settings, convert.

Where to go next

- Convert several documents at once from a terminal: Convert documents from the command line.
- Understand what happens inside the sandbox: How Dangerzone works.
- See the full list of supported document formats.

Convert documents from the command line

Dangerzone ships a command-line tool, `dangerzone-cli`, next to the graphical application. In this tutorial you will use it to convert a single document, then a batch of documents with OCR, and archive the originals. This is handy when you have many files, or when you want to automate the conversion process.

You need Dangerzone installed (see the installation guide) and a terminal. The examples use a folder called `inbox` with a couple of PDFs and office documents in it. Any files from the supported formats will do.

1. Find the command

The command lives in different places depending on your platform:

Linux

`dangerzone-cli` is on your `PATH`:

```
$ dangerzone-cli --version
0.11.0
```

> **Note**
>
> With the slim `dangerzone` package, the CLI does not download the sandbox on its own. Either start the graphical application once and accept the download, or run `dangerzone-image upgrade` first.

macOS

The tool is inside the application bundle:

```
$ /Applications/Dangerzone.app/Contents/MacOS/dangerzone-cli --version
0.11.0
```

Define an alias for the rest of this tutorial:

```
alias dangerzone-cli=/Applications/Dangerzone.app/Contents/MacOS/dangerzone-cli
```

Windows

The tool is in the installation folder. In a terminal:

```
& 'C:\Program Files\Dangerzone\dangerzone-cli.exe' --version
```

> The examples below use `dangerzone-cli` for brevity. Prefix it with the full path, or add `C:\Program Files\Dangerzone` to your `PATH`.

2. Convert one document

```
$ cd inbox
$ dangerzone-cli invoice.pdf
```

Dangerzone prints a banner, starts the sandbox, and shows the progress of each stage. On macOS and Windows, the first run also starts the Podman machine, a small virtual machine that runs the sandbox, which takes a little longer.

When it finishes, `invoice-safe.pdf` sits next to `invoice.pdf`. The original is untouched: unlike the graphical application, the CLI does not move originals unless you ask it to (see "archive" below).

3. Choose the output name

Pass `--output-filename` to control where the safe PDF goes. This only works with a single input file:

```
$ dangerzone-cli invoice.pdf --output-filename ~/Documents/invoice-clean.pdf
```

4. Convert a batch with OCR

Give the command several files, and add `--ocr-lang` with a language code so that the safe PDFs get a searchable text layer:

```
$ dangerzone-cli --ocr-lang eng report.docx slides.pptx scan.jpg
```

Conversions run one after the other and each one reports its own progress. The output files are `report-safe.pdf`, `slides-safe.pdf`, and `scan-safe.pdf`.

If you pass a language code Dangerzone doesn't know, it prints the list of valid codes and exits. `eng`, `fra`, `deu`, `spa`, and `ara` are examples of valid codes.

5. Archive the originals

Add `--archive` to move each original into an `unsafe/` subdirectory after a successful conversion, exactly like the graphical application does by default:

```
$ dangerzone-cli --archive --ocr-lang eng *.pdf
$ ls
report-safe.pdf  scan-safe.pdf  unsafe/
$ ls unsafe
report.pdf  scan.pdf
```

6. Keep the machine running between batches (macOS and Windows)

On macOS and Windows, Dangerzone stops the Podman machine after each invocation. If you plan to run several commands in a row, pass `--linger` so the machine stays up, then stop it yourself when you are done:

```
$ dangerzone-cli --linger first-batch/*.pdf
$ dangerzone-cli --linger second-batch/*.pdf
$ dangerzone-machine stop
```

What you learned

You can now convert one or many documents from a terminal, name the output, add OCR, and archive the originals. The dangerzone-cli reference lists every option, including `--debug` for gVisor logs when something goes wrong.

Where to go next

- Update the sandbox image by hand, or install it on an air-gapped machine: Update the sandbox image.
- Manage the Podman machine on macOS and Windows: dangerzone-machine reference.

Run Dangerzone from source

This tutorial walks you through a first development setup: you will clone the repository, install the dependencies, run the application and its test suite from the source tree, and make a small change to see it live. It follows a single path, on Ubuntu or Debian, so that every step is predictable. The build from source how-to guides cover the other platforms and the packaging steps.

You need a Debian or Ubuntu machine (a virtual machine will only work if you can do nested virtualization), `sudo` access, about 5 GB of free disk space, and an internet connection.

1. Install the system dependencies

```
sudo apt install -y podman dh-python build-essential make libqt6gui6 \
    pipx python3 python3-dev
```

Podman runs the sandbox. The Qt library is needed by the graphical application. `pipx` is used in the next step.

> **Ubuntu 22.04 (Jammy) users**
>
> The `conmon` version that Podman uses and Ubuntu Jammy ships has a bug that gets triggered by Dangerzone (more details in #685). Install a patched `conmon` before continuing, as described in the Ubuntu and Debian build guide.

2. Install Poetry

Dangerzone manages its Python dependencies with Poetry:

```
pipx ensurepath
pipx install poetry
pipx inject poetry poetry-plugin-export
```

Close and reopen your terminal so that `poetry` is on your `PATH`, then check:

```
$ poetry --version
Poetry (version 2.x)
```

3. Clone the repository and install the Python dependencies

```
git clone https://github.com/freedomofpress/dangerzone/
cd dangerzone
poetry install
```

> **Note**
>
> If Poetry prompts for your keyring, disable it with `keyring --disable` and run `poetry install` again. This is a known Poetry issue.

4. Download the assets and the sandbox image

Dangerzone depends on a few binaries and external resources. Fetch them with:

```
poetry run mazette install
```

Then fetch the latest sandbox image and store it where the development build expects it:

```
export DANGERZONE_DEV=1
poetry run dangerzone-image prepare-archive --output share/container.tar
```

`DANGERZONE_DEV=1` tells Dangerzone that it runs from a source tree, so it looks for resources under `share/` instead of the system paths. Keep it set for the rest of this tutorial.

5. Run the application

Start the graphical application:

```
poetry run dangerzone
```

The main window appears. Convert a document if you like, following Convert your first document, then close the window.

The command-line tool works the same way:

```
poetry run dangerzone-cli --help
```

6. Run the tests

```
poetry run make test
```

The suite contains tests for the CLI, the GUI (headless), and the isolation providers. It takes a few minutes and ends with a summary of passed tests. Some tests convert sample documents from `tests/test_docs/` through the real sandbox, so Podman has to work on your machine for the suite to pass.

7. Make a change and see it

Open `dangerzone/gui/main_window.py` and find the label of the button that selects documents:

```
self.dangerous_doc_button = QtWidgets.QPushButton("Select suspicious documents ...")
```

Change the text to anything you would like, save, and start the application again with `poetry run dangerzone`. The button now shows your text: the application runs straight from the source tree, so there is no build step between an edit and a run.

Revert the change with `git checkout dangerzone/gui/main_window.py` when you are done.

What you learned

You now have a working development environment: `poetry run dangerzone` starts the app, `poetry run make test` runs the tests, and `DANGERZONE_DEV=1` points Dangerzone at the resources in `share/`.

Where to go next

- Lint and format like CI does with `poetry run make lint` and `poetry run make fix`.
- Build a `.deb` package from your checkout: Build from source on Ubuntu and Debian.
- Test in a containerized environment for another distribution (Fedora, or older Ubuntu and Debian): Use the containerized dev environments.
- Read how the pieces fit together: How Dangerzone works.

Guides

Install

Pick your operating system to see how to install Dangerzone:

macOS

- Download Dangerzone 0.11.1 for Mac (Apple Silicon CPU)
- Download Dangerzone 0.11.1 for Mac (Intel CPU)

You can also install Dangerzone for Mac using Homebrew: `brew install --cask dangerzone`

> **See which versions of macOS are supported**
>
> We support the releases of macOS that are within Apple's servicing timeline, usually meaning security updates for the latest 3 releases.

Windows

- Download Dangerzone 0.11.1 for Windows

You can also install Dangerzone for Windows using Winget:

```
winget install FreedomofthePressFoundation.Dangerzone
```

> **See which versions of Windows are supported**
>
> We generally support Windows releases that are within Microsoft's servicing timeline. That means that a recent release of Windows 10 or Windows 11 is required. In addition, you need to have hardware virtualization enabled (to be more specific, on x64, WSL requires build 18362 or later, and 19041 or later is required for arm64 systems). If you are running Windows in a virtual machine, the VM must support nested virtualization. These requirements come from Podman, which Dangerzone uses under the hood.

Ubuntu, Debian

See the list of supported Debian and Ubuntu versions

Dangerzone is available for:

- Ubuntu 26.10 (stonking)
- Ubuntu 26.04 (resolute)
- Ubuntu 25.10 (questing)
- Ubuntu 24.04 (noble)
- Ubuntu 22.04 (jammy)
- Debian 14 (forky)
- Debian 13 (trixie)
- Debian 12 (bookworm)

Our "support rule" is that we support Ubuntu and Debian releases that are still within their respective servicing timelines, with a few twists:

- Ubuntu: We follow upstream support with an extra cutoff date. No support for versions prior to the second oldest LTS release.
- Debian: We support the last two stable releases.

Add our repository following these instructions:

Download the GPG key for the repo:

```
sudo apt-get update && sudo apt-get install -y gpg ca-certificates
sudo mkdir -p /etc/apt/keyrings
sudo gpg --keyserver hkps://keys.openpgp.org \
    --no-default-keyring --no-permission-warning --homedir $(mktemp -d) \
    --keyring gnupg-ring:/etc/apt/keyrings/fpf-apt-tools-archive-keyring.gpg \
    --recv-keys DE28AB241FA48260FAC9B8BAA7C9B38522604281
sudo chmod +r /etc/apt/keyrings/fpf-apt-tools-archive-keyring.gpg
```

Add the URL of the repo in your APT sources:

```
. /etc/os-release
echo "deb [signed-by=/etc/apt/keyrings/fpf-apt-tools-archive-keyring.gpg] \
    https://packages.freedom.press/apt-tools-prod ${VERSION_CODENAME?} main" \
    | sudo tee /etc/apt/sources.list.d/fpf-apt-tools.list
```

Install Dangerzone:

```
sudo apt update
sudo apt install -y dangerzone

# Alternatively, use dangerzone-full if you prefer to install dangerzone
# with its container bundled (larger package, but no download at first use)
sudo apt install -y dangerzone-full
```

Expand this section for a security notice on third-party Debian repos

This section follows the official instructions on configuring third-party Debian repos.

To mitigate a class of attacks against our APT repo (e.g., injecting packages signed with an attacker key), we add an additional step in our instructions to verify the downloaded GPG key against its fingerprint.

Fedora

See the list of supported Fedora versions

Dangerzone is available for:

- Fedora 45
- Fedora 44
- Fedora 43

We support Fedora releases that are still within their servicing timeline (we follow upstream support).

Type the following commands in a terminal:

```
sudo dnf install 'dnf-command(config-manager)'
sudo dnf config-manager addrepo --from-repofile=https://packages.freedom.press/yum-tools-prod/
dangerzone/dangerzone.repo
sudo dnf install dangerzone

# Alternatively, use dangerzone-full if you prefer to install dangerzone
# with its container bundled (larger package, but no download at first use)
sudo dnf install dangerzone-full
```

Verifying Dangerzone GPG key

Importing GPG key 0x22604281: ... Is this ok [y/N]:

After some minutes of running the above command (depending on your internet speed) you'll be asked to confirm the fingerprint of our signing key. This is to make sure that in the case our servers are compromised your computer stays safe. It should look like this:

```
--------------------------------------------------------------------------------
Total                                           389 kB/s | 732 MB     32:07
Dangerzone repository                           3.8 MB/s | 3.8 kB     00:00
Importing GPG key 0x22604281:
 Userid     : "Dangerzone Release Key <dangerzone-release-key@freedom.press>"
 Fingerprint: DE28 AB24 1FA4 8260 FAC9 B8BA A7C9 B385 2260 4281
 From       : /etc/pki/rpm-gpg/RPM-GPG-dangerzone.pub
Is this ok [y/N]:
```

> **Note**: If it does not show this fingerprint confirmation or the fingerprint does not match, it is possible that our servers were compromised. Be distrustful and reach out to us.

The `Fingerprint` should be `DE28 AB24 1FA4 8260 FAC9 B8BA A7C9 B385 2260 4281`. For extra security, you should confirm it matches the one at the bottom of our website (dangerzone.rocks) and our Mastodon account bio.

After confirming that it matches, type `y` (for yes) and the installation should proceed.

Fedora Atomic (Silverblue, Kinoite, etc.)

Warning

This distribution is not officially supported by the Dangerzone team. Please proceed at your own risk, and only if you know what you're doing.

Type the following commands in a terminal:

```
curl -o - https://packages.freedom.press/yum-tools-prod/dangerzone/dangerzone.repo \
    | sudo tee /etc/yum.repos.d/dangerzone.repo > /dev/null
rpm-ostree install dangerzone
# Alternatively, use dangerzone-full if you prefer to install dangerzone
# with its container bundled (larger package, but no download at first use)
rpm-ostree install dangerzone-full
```

After the above command completes, restart your computer to complete the installation.

Qubes OS

> **Qubes support is still in beta**
>
> This section is for the beta version of native Qubes support. If you want to try out the stable Dangerzone version (which uses containers instead of virtual machines for isolation), please follow the Fedora or Debian instructions and adapt them as needed.
>
> If you followed these instructions before October 25, 2023, please read this security advisory. This notice will be removed with the 1.0.0 release of Dangerzone.

> **You might want to use a different template**
>
> This section will install Dangerzone in your **default template** (`fedora-43` as of writing this). If you want to install it in a different one, make sure to replace `fedora-43` with the template of your choice.

The following steps must be completed once. Make sure you run them in the specified qubes.

Overview of the qubes you'll create:

qube	type	purpose
dz-dvm	app qube	offline disposable template for performing conversions

In `dom0`:

Create a **disposable**, offline app qube (`dz-dvm`), based on your default template. This will be the qube where the documents will be sanitized:

```
qvm-create --class AppVM --label red --template fedora-43 \
    --prop netvm="" --prop template_for_dispvms=True \
    --prop default_dispvm='' dz-dvm
```

Add an RPC policy (`/etc/qubes/policy.d/50-dangerzone.policy`) that will allow launching a disposable qube (`dz-dvm`) when Dangerzone converts a document, with the following contents:

```
dz.Convert         *       @anyvm       @dispvm:dz-dvm  allow
```

In the `fedora-43` template

Install Dangerzone:

```
sudo dnf install 'dnf-command(config-manager)'
sudo dnf config-manager addrepo --from-repofile=https://packages.freedom.press/yum-tools-prod/
dangerzone/dangerzone.repo
sudo dnf install dangerzone-qubes
```

While Dangerzone gets installed, you will be prompted to accept a signing key. Expand the instructions in the Verifying Dangerzone GPG key section to verify the key.

Finally, shut down the template and restart the qubes where you want to use Dangerzone. Go to "Qube Settings" -> choose the "Applications" tab, click on "Refresh applications", and then move "Dangerzone" from the "Available" column to "Selected".

You can now launch Dangerzone from the list of applications for your qube, and pass it a file to sanitize.

Tails

You can follow installation instructions on the Tails documentation to install Dangerzone on Tails (it is not included by default, at least for now.)

You can read more about the supported platforms on the dedicated section.

If you prefer to build Dangerzone from source, follow the run Dangerzone from source tutorial.

Verifying PGP signatures

You can verify that the package you download is legitimate and hasn't been tampered with by verifying its PGP signature. For Windows and macOS, this step is optional and provides defense in depth: the Dangerzone binaries include operating system-specific signatures, and you can just rely on those alone if you'd like.

Obtaining signing key

Our binaries are signed with a PGP key owned by Freedom of the Press Foundation:

- Name: Dangerzone Release Key
- PGP public key fingerprint `DE28 AB24 1FA4 8260 FAC9 B8BA A7C9 B385 2260 4281`
    - You can download this key from the keys.openpgp.org keyserver.

(You can also cross-check this fingerprint with the fingerprint in our Mastodon page and the fingerprint in the footer of our official site)

You must have GnuPG installed to verify signatures. For macOS you probably want GPGTools, and for Windows you probably want Gpg4win.

Signatures

Our GitHub Releases page hosts the following files:

- Windows installer (`Dangerzone-<version>.msi`)
- macOS archives (`Dangerzone-<version>-<arch>.dmg`)
- Container images (`container-<version>-<arch>.tar`)

All these files are accompanied by signatures (as `.asc` files). We'll explain how to verify them below, using `0.6.1` as an example.

Verifying

Once you have imported the Dangerzone release key into your GnuPG keychain, downloaded the binary and `.asc` signature, you can verify the binary in a terminal like this:

For the Windows binary:

```
gpg --verify Dangerzone-0.6.1.msi.asc Dangerzone-0.6.1.msi
```

For the macOS binaries (depending on your architecture):

```
gpg --verify Dangerzone-0.6.1-arm64.dmg.asc Dangerzone-0.6.1-arm64.dmg
gpg --verify Dangerzone-0.6.1-i686.dmg.asc Dangerzone-0.6.1-i686.dmg
```

For the container images:

```
gpg --verify container-0.6.1-i686.tar.asc container-0.6.1-i686.tar
```

For the source package:

```
gpg --verify dangerzone-0.6.1.tar.gz.asc dangerzone-0.6.1.tar.gz
```

We also hash all the above files with SHA-256, and provide a list of these hashes as a separate file (`checksums-0.6.1.txt`). This file is signed as well, and the signature is embedded within it. You can download this file and verify it with:

```
gpg --verify checksums.txt
```

The expected output looks like this:

```
gpg: Signature made Mon Apr 22 09:29:22 2024 PDT
gpg:                using RSA key 04CABEB5DD76BACF2BD43D2FF3ACC60F62EA51CB
gpg: Good signature from "Dangerzone Release Key <dangerzone-release-key@freedom.press>" [unknown]
gpg: WARNING: This key is not certified with a trusted signature!
gpg:          There is no indication that the signature belongs to the owner.
Primary key fingerprint: DE28 AB24 1FA4 8260 FAC9  B8BA A7C9 B385 2260 4281
     Subkey fingerprint: 04CA BEB5 DD76 BACF 2BD4  3D2F F3AC C60F 62EA 51CB
```

If you don't see `Good signature from`, there might be a problem with the integrity of the file (malicious or otherwise), and you should not install the package.

The `WARNING:` shown above is not a problem with the package. It only means you haven't defined a level of "trust" for Dangerzone's PGP key.

If you want to learn more about verifying PGP signatures, the guides for Qubes OS and the Tor Project may be useful.

Update Dangerzone

Dangerzone gets updates to improve its features *and* to fix problems, so updating may be the simplest path to resolving an issue you are experiencing.

Check whether you are up to date

1. Check which version of Dangerzone you are currently using: run Dangerzone, then look for a series of numbers to the right of the logo within the app. The format of the numbers will look similar to `0.4.1`.
2. Now find the latest available version of Dangerzone: go to the download page. Look for the version number displayed.
3. If the version on the Dangerzone download page is higher than the version of your installed app, you should consider updating.

On Windows and macOS, if you enabled update checks, Dangerzone also shows a green dot on the hamburger menu in the top-right corner when a new release is out. Click it and choose **New version available** to see the changelog.

If you're curious about how that works, have a look at the update notifications section.

Install the update

macOS

Download the new `.dmg` from the download page and drag Dangerzone into your Applications folder again, as you did for the initial installation. Your settings are kept.

If you installed Dangerzone with Homebrew, upgrade it the same way:

```
brew upgrade --cask dangerzone
```

Windows

Download the new `.msi` from the download page and run it, as you did for the initial installation. Your settings are kept.

If you installed Dangerzone with Winget, upgrade it the same way:

```
winget upgrade FreedomofthePressFoundation.Dangerzone
```

Ubuntu and Debian

```
sudo apt update
sudo apt upgrade dangerzone
```

> Replace `dangerzone` with `dangerzone-full` if that is the package you installed.

Fedora

```
sudo dnf upgrade dangerzone
```

Replace `dangerzone` with `dangerzone-full` if that is the package you installed. On Fedora Atomic, run `rpm-ostree upgrade` and reboot.

Qubes OS

Update the template where you installed `dangerzone-qubes` with `sudo dnf upgrade dangerzone-qubes`, then shut down the template and restart the qubes that use it.

Updating the sandbox without updating Dangerzone

The sandbox image updates independently from the application. If you enabled automatic updates, Dangerzone fetches new sandbox images on its own. To trigger it by hand, or on an air-gapped machine, see Update the sandbox image.

Update check errors

When update checks are enabled, Dangerzone contacts GitHub at most once every 12 hours to learn about new releases. If a check fails, a red dot shows up on the hamburger menu in the top-right corner. Click it and choose **Update error** to read the error message.

Most of the time, this means that your computer is not connected to the Internet, or that GitHub is experiencing an outage. Dangerzone tries again on its own 12 hours later, and the error goes away after the next successful check. In the meantime, you can check by hand whether you are up to date.

If the error keeps coming back, or if you are unsure about the message you see, please report it.

If your computer is never online, uncheck **Check for updates** in the hamburger menu to turn the checks off, and follow our releases some other way: on Mastodon, or with the **Watch** button of our GitHub repository. To update the sandbox on such a machine, see Installing image updates to air-gapped environments.

Update the sandbox

Dangerzone has a mechanism to auto-update the secure sandbox used for document conversion.

This mechanism allows us to ship security fixes without having to do a full-blown release, shortening the time between when security patches are out and when they are used.

This increases the security of the conversion process dramatically, making it harder for an attacker to rely on known and patched exploits in our stack.

In order to ensure the sandbox image is trusted, we sign it with Cosign, which is part of the greater Sigstore ecosystem, and verify it against a key distributed in the Dangerzone application.

Install updates

Checking for updates is done automatically when the setting is turned on. In addition, it's possible to check if a new sandbox image has been released and update your local installation with the following command:

```
dangerzone-image upgrade
```

Verify locally

You can verify that the image you have locally matches the stored signatures, and that these have been signed with a trusted public key. This is not a required step, and will be done automatically by the Dangerzone software on subsequent runs, so this command is mainly provided as a convenience.

```
dangerzone-image verify-local
```

The public key used to verify the container signatures is shipped in `share/freedomofpress-dangerzone.pub`, signed by our main Dangerzone release key. You can verify this signature using `gpg`.

Checking attestations

Each night, new images are built and pushed to the container registry, alongside a provenance attestation. This lets anybody check that the image was originally built by GitHub CI runners, from a defined source repository (in our case `freedomofpress/dangerzone`).

To verify the attestations against our expectations, clone the GitHub repo that builds our container images, and use the following command:

```
git clone https://github.com/freedomofpress/dangerzone-image
cd dangerzone-image/
```

```
uvx mazette install
uv run image verify-attestation ghcr.io/freedomofpress/dangerzone/v1@sha256:...
```

In case of success, it will report back:

```
🎉 Successfully verified image
'ghcr.io/freedomofpress/dangerzone/v1:<tag>@sha256:<digest>'
and its associated claims:
- ✅ SLSA Level 3 provenance
- ✅ GitHub repo: freedomofpress/dangerzone-image
- ✅ GitHub actions workflow: <workflow>
```

Installing image updates to air-gapped environments

To install a container image to an environment without a network connection, three steps are required:

1. Prepare the archive
2. Transfer the archive to the air-gapped system
3. Install the archive on the air-gapped system

This archive will contain all the needed material to verify that the new container image has been signed and is valid.

On the machine on which you prepare the packages (of course, adapt to the architecture you want to target):

```
dangerzone-image prepare-archive --arch amd64
```

On the airgapped machine, copy the file and run the following command:

```
dangerzone-image load-archive dangerzone-amd64.tar
```

Configuring the verification material

Dangerzone bundles and pins the public key of the Rekor service, which powers the transparency log of Sigstore signatures.

If Sigstore maintainers decide to rotate this key, a new Dangerzone version will be released, bundled with the new key. Power users can specify an updated key in the meantime, by fetching the latest Rekor public key with:

```
cosign initialize
cat ~/.sigstore/root/tuf-repo-cdn.sigstore.dev/targets/trusted_root.json  \
    | jq -r .tlogs[0].publicKey.rawBytes \
    | base64 -d \
    | openssl pkey -pubin > rekor.pub
```

And set it with the following environment variable:

```
export SIGSTORE_REKOR_PUBLIC_KEY=rekor.pub
dangerzone
```

Using Podman Desktop

Dangerzone uses the system-installed Podman on Linux, and embeds Podman on Windows and macOS.

If you wish to use Podman Desktop with a specific configuration, follow these steps. These instructions might work for other custom runtimes, but bear in mind that only Podman is currently supported.

On macOS

To set the container runtime to Podman Desktop, use this command:

```
/Applications/Dangerzone.app/Contents/MacOS/dangerzone-cli --set-container-runtime /opt/podman/bin/
podman
```

To revert back to the default behavior, pass the `default` value:

```
/Applications/Dangerzone.app/Contents/MacOS/dangerzone-cli --set-container-runtime default
```

On Windows

To set the container runtime to Podman desktop, use this command:

```
C:\"Program Files"\Dangerzone\dangerzone-cli.exe --set-container-runtime \path\to\your\runtime
```

To revert back to the default behavior, pass the `default` value:

```
C:\"Program Files"\Dangerzone\dangerzone-cli.exe --set-container-runtime podman
```

FAQ

I've encountered the following error: `Error: nomap is only supported in rootless mode`

This means that Podman does not run in rootless mode. You can switch to rootless mode with these steps:

```
podman machine stop
podman machine set --rootful=false
podman machine start
```

Report an issue

This guide explains where and how to report a problem with Dangerzone.

Security issues

Please do not report security vulnerabilities through public GitHub issues. If you believe you have found a vulnerability, follow the security policy: it lists the private channels, what we consider in scope, and the timelines we commit to.

Check the obvious first

1. Make sure you run the latest version. Many issues are fixed by updating. See Update Dangerzone.
2. Search the existing issues for the error message you see. Add a comment to an existing issue rather than opening a duplicate.
3. Skim the FAQ.

Collect the details

A useful report contains:

- The Dangerzone version (shown next to the logo in the application, or with `dangerzone-cli --version`).
- Your operating system and version, and how you installed Dangerzone (installer, Homebrew, Winget, `.deb`, `.rpm`, Qubes, Tails).
- What you did, what you expected, and what happened instead.
- The exact error message. The graphical application shows a log window on failures, and its contents are what we need.

Details about the sandbox

Oftentimes, issues are related to Podman (the tool we use to run the conversion sandbox). In these cases, it's very useful to have a lot of details about it.

Windows

Please copy and paste the following commands in your terminal, and provide us with the output:

```
'C:\Program Files\Dangerzone\dangerzone-machine.exe' start
'C:\Program Files\Dangerzone\dangerzone-machine.exe' --log-level debug raw version
'C:\Program Files\Dangerzone\dangerzone-machine.exe' --log-level debug raw info -f json
'C:\Program Files\Dangerzone\dangerzone-machine.exe' --log-level debug raw images
'C:\Program Files\Dangerzone\dangerzone-machine.exe' --log-level debug raw run hello-world
```

Additionally, if you tried to do a conversion, paste the output of this command:

```
'C:\Program Files\Dangerzone\dangerzone-cli.exe' --debug the-document.pdf
```

macOS

Please copy and paste the following commands in your terminal, and provide us with the output:

```
/Applications/Dangerzone.app/Contents/MacOS/dangerzone-machine start
/Applications/Dangerzone.app/Contents/MacOS/dangerzone-machine --log-level debug raw version
/Applications/Dangerzone.app/Contents/MacOS/dangerzone-machine --log-level debug raw info -f 'js
on'
/Applications/Dangerzone.app/Contents/MacOS/dangerzone-machine --log-level debug raw images
/Applications/Dangerzone.app/Contents/MacOS/dangerzone-machine --log-level debug raw run hello-
world
```

Additionally, if you tried to do a conversion, paste the output of this command:

```
/Applications/Dangerzone.app/Contents/MacOS/dangerzone-cli --debug the-document.pdf
```

Linux

Please copy and paste the following commands in your terminal, and provide us with the output:

```
podman version
podman info -f 'json'
podman images
podman run hello-world
```

Additionally, if you tried to do a conversion, paste the output of this command:

```
dangerzone-cli --debug the-document.pdf
```

Don't attach the suspicious document

The document that failed to convert may be the problem, and it may be malicious. Don't attach it to a public issue. Describe it instead (format, size, how it was produced).

Open the issue

Open a new issue in the GitHub tracker with the details above. We triage issues regularly and may ask follow-up questions, so keep an eye on notifications. If you don't have or don't want to create a Github account to report an issue, you can also contact us via email at `support@dangerzone.rocks`.

For problems with the sandbox image itself (a document format that doesn't convert, a missing font, a CVE in a bundled tool), the dangerzone-image repository is the right place. When in doubt, report to the main repository and we will move it.

Other channels

- Release announcements and short updates are posted on Mastodon.
- General questions can also go to the issue tracker or in our discussion board.

Development environment

Pick your platform to set up a development environment:

Ubuntu, Debian

Install dependencies:

> **Expand this section if you are on Ubuntu 22.04 (Jammy).**
>
> The `conmon` version that Podman uses and Ubuntu Jammy ships, has a bug that gets triggered by Dangerzone (more details in https://github.com/freedomofpress/dangerzone/issues/685). If you want to run Dangerzone from source, you are advised to install a patched `conmon` version. A simple way to do so is to enable our apt-tools-prod repo, just for the `conmon` package:
>
> ```
> sudo cp ./dev_scripts/apt-tools-prod.sources /etc/apt/sources.list.d/
> sudo cp ./dev_scripts/apt-tools-prod.pref /etc/apt/preferences.d/
> ```
>
> The `conmon` package provided in the above repo was built with the following instructions. Alternatively, you can install a `conmon` version higher than `v2.0.25` from any repo you prefer.

```
sudo apt install -y podman dh-python build-essential make libqt6gui6 \
    pipx python3 python3-dev
```

Install Poetry using `pipx` (recommended) and add it to your `$PATH`:

(See also a list of alternative installation methods)

```
pipx ensurepath
pipx install poetry
pipx inject poetry poetry-plugin-export
```

After this, restart the terminal window so that the `poetry` command is in your `$PATH`.

Clone this repository:

```
git clone https://github.com/freedomofpress/dangerzone/
```

Change to the `dangerzone` folder, and install the poetry dependencies:

> **Note**: due to an issue with poetry, if it prompts for your keyring, disable the keyring with `keyring --disable` and run the command again.

```
cd dangerzone
poetry install
```

Dangerzone depends on some assets (think binaries and other resources) that should be downloaded in order to run. This can be done with the following command:

```
poetry run mazette install
```

Run the following command to download the latest container image, or use a local one:

```
export DANGERZONE_DEV=1
poetry run dangerzone-image prepare-archive --output share/container.tar
```

Run from source tree:

```
# run the CLI
poetry run dangerzone-cli --help

# run the GUI
poetry run dangerzone
```

Create a .deb:

```
./install/linux/build-deb.py
```

This builds both the `dangerzone` (slim) and `dangerzone-full` packages in a single run. The slim package does not contain the `container.tar` image (it will be downloaded on the first run), while `dangerzone-full` bundles the container image for offline or air-gapped installations.

> **Note**: `container.tar` must be present in `share/` before running this command, as it is required for the `dangerzone-full` package. See the instructions above for how to download or build it.

Fedora

Install dependencies:

```
sudo dnf install -y rpm-build podman python3 python3-devel python3-poetry-core \
    pipx qt6-qtbase-gui
```

> **Note**: The default Python version that ships with Fedora 45 (3.15) is not compatible with PySide6 from PyPI, which requires Python 3.14 or earlier. If you are on Fedora 45, install Python 3.14 with `sudo dnf install -y python3.14`, and run `poetry env use python3.14` in the `dangerzone` folder, before running `poetry install`.

Install Poetry using `pipx`:

```
pipx install poetry
pipx inject poetry
```

Clone this repository:

```
git clone https://github.com/freedomofpress/dangerzone/
```

Change to the `dangerzone` folder, and install the poetry dependencies:

> **Note**: due to an issue with poetry, if it prompts for your keyring, disable the keyring with `keyring --disable` and run the command again.

```
cd dangerzone
poetry install
```

Dangerzone depends on some assets (think binaries and other resources) that should be downloaded in order to run. This can be done with the following command:

```
poetry run mazette install
```

Run the following command to download the latest container image, or use a local one:

```
export DANGERZONE_DEV=1
poetry run dangerzone-image prepare-archive --output share/container.tar
```

Run from source tree:

```
# run the CLI
poetry run dangerzone-cli --help

# run the GUI
poetry run dangerzone
```

> **Note**
>
> Prefer running the following command in a Fedora development environment, created by `./dev_script/env.py`.

Create a .rpm:

```
./install/linux/build-rpm.py
```

This builds the `dangerzone` package, which doesn't contain the `container.tar` image (it will be downloaded on the first run).

To build the `dangerzone-full` package with the container bundled:

```
./install/linux/build-rpm.py --full
```

Qubes OS

> ⚠️ Native Qubes support is in beta stage, so the instructions below require switching between qubes, and are subject to change.
>
> If you want to build Dangerzone on Qubes and use containers instead of disposable qubes, please follow the instructions of Fedora / Debian instead.

Initial Setup

The following steps must be completed once. Make sure you run them in the specified qubes.

Overview of the qubes you'll create:

qube	type	purpose
dz	app qube	Dangerzone development
dz-dvm	app qube	offline disposable template for performing conversions
fedora-43-dz	template	template for the other two qubes

In `dom0`:

The following instructions require typing commands in a terminal in dom0.

1. Create a new Fedora **template** (`fedora-43-dz`) for Dangerzone development:

   ```
   qvm-clone fedora-43 fedora-43-dz
   ```

   > 💡 Alternatively, you can use your base Fedora 43 template in the following instructions. In that case, skip this step and replace `fedora-43-dz` with `fedora-43` in the steps below.

2. Create an offline disposable template (app qube) called `dz-dvm`, based on the `fedora-43-dz` template. This will be the qube where the documents will be sanitized:

   ```
   qvm-create --class AppVM --label red --template fedora-43-dz \
       --prop netvm="" --prop template_for_dispvms=True \
       --prop default_dispvm='' dz-dvm
   ```

3. Create an **app** qube (`dz`) that will be used for Dangerzone development and initiating the sanitization process:

   ```
   qvm-create --class AppVM --label red --template fedora-43-dz dz
   qvm-volume resize dz:private $(numfmt --from=auto 20Gi)
   ```

   > 💡 Alternatively, you can use a different app qube for Dangerzone development. In that case, replace `dz` with the qube of your choice in the steps below.

> In the commands above, we also resize the private volume of the `dz` qube to 20GiB, since you may need some extra storage space when developing on Dangerzone (e.g., for container images, Tesseract data, and Python virtualenvs).

4. Add an RPC policy (`/etc/qubes/policy.d/50-dangerzone.policy`) that will allow launching a disposable qube (`dz-dvm`) when Dangerzone converts a document, with the following contents:

```
dz.Convert          *       @anyvm       @dispvm:dz-dvm  allow
dz.ConvertDev       *       @anyvm       @dispvm:dz-dvm  allow
```

In the `dz` app qube

In the following steps you'll setup the development environment and install a dangerzone build. This will make development faster, since the server code is loaded dynamically on each run, instead of having to build and install a server package every time you want to test it.

1. Follow the Fedora installation instructions up until `poetry run mazette install`.
2. Build the Dangerzone RPM packages for Qubes:

```
./install/linux/build-rpm.py --qubes
```

3. Copy the produced `.rpm` files into `fedora-43-dz`:

```
qvm-copy ./dist/*.rpm
```

In the `fedora-43-dz` template

1. Install the `.rpm` packages you just copied, in order to get the Dangerzone dependencies:

```
sudo dnf install ~/QubesIncoming/dz/*.rpm
```

2. Shutdown the `fedora-43-dz` template.

Developing Dangerzone

From here on, developing Dangerzone is similar to Fedora. The only differences are that you need to set the environment variable `QUBES_CONVERSION=1` when you wish to test the Qubes conversion. Run the following commands on the `dz` development qube:

```
export DANGERZONE_DEV=1 QUBES_CONVERSION=1

# run the CLI
poetry run dangerzone-cli --help
```

```
# run the GUI
poetry run dangerzone
```

And when creating a `.rpm` you'll need to enable the `--qubes` flag.

> **Note**
>
> Prefer running the following command in a Fedora development environment, created by `./dev_script/env.py`.

```
./install/linux/build-rpm.py --qubes
```

> **Tip**
>
> For faster changes to the server side components, you can let Dangerzone know about the location of `dangerzone-image` repo:
>
> ```
> export DANGERZONE_INSECURE_CONVERTER_PATH=~/dangerzone-image/src
> ```
>
> From there on, you can make changes in the `dangerzone-image` repo, and they will be mirrored to the disposable qube through the `dz.ConvertDev` RPC call.
>
> The only reason to build a new Qubes RPM and install it in the `fedora-43-dz` template for development is if:
>
> 1. The project requires new server-side components.
> 2. The code for `qubes/dz.ConvertDev` needs to be updated.

macOS

Install the latest version of Python 3.14 from python.org, and make sure `/Library/Frameworks/Python.framework/Versions/3.14/bin` is in your `PATH`.

Clone this repository:

```
git clone https://github.com/freedomofpress/dangerzone/
cd dangerzone
```

Install Python dependencies:

```
python3 -m pip install poetry
poetry install
```

Install Homebrew dependencies:

```
brew install create-dmg
```

Dangerzone depends on some assets (think binaries and other resources) that should be downloaded in order to run. This can be done with the following command:

```
poetry run mazette install
```

Make sure to set the `DANGERZONE_DEV` env variable for development:

```
export DANGERZONE_DEV=1
```

Run the following command to download the latest container image, or use a local one:

```
poetry run dangerzone-image prepare-archive --output share/container.tar
```

Run from source tree:

```
# run the CLI
poetry run dangerzone-cli --help

# run the GUI
poetry run dangerzone
```

To create an app bundle, use the `build-app.py` script:

```
poetry run ./install/macos/build-app.py
```

If you want to build for distribution, you'll need a codesigning certificate, and then run:

```
poetry run ./install/macos/build-app.py --with-codesign
```

The output is in the `dist` folder.

Windows

Install the latest version of Python 3.14 (64-bit) from python.org. Make sure to check the "Add Python 3.14 to PATH" checkbox on the first page of the installer.

Install Microsoft Visual C++ 14.0 or greater. Get it with "Microsoft C++ Build Tools" and make sure to select "Desktop development with C++" when installing.

Install git from here.

Install poetry.

Open Windows "Terminal" application and run all remaining commands there.

```
python -m pip install poetry
```

Clone this repository:

```
git clone https://github.com/freedomofpress/dangerzone/
```

Change to the `dangerzone` folder, and install the poetry dependencies:

```
cd dangerzone
poetry install
```

Dangerzone depends on some assets (think binaries and other resources) that should be downloaded in order to run. This can be done with the following command:

```
poetry run mazette install
```

Run the following command to download the latest container image, or use a local one:

```
$Env:DANGERZONE_DEV = 1
poetry run dangerzone-image prepare-archive --output share\container.tar
```

After that you can launch dangerzone during development with:

```
# run the CLI
$Env:DANGERZONE_DEV = 1; poetry run dangerzone-cli --help

# run the GUI
$Env:DANGERZONE_DEV = 1; poetry run dangerzone
```

If you want to build the Windows installer

Install .NET SDK version 6 or later. Then, open a terminal and install the latest version of WiX Toolset .NET tool **v5** with:

```
dotnet tool install --global wix --version 5.0.2
```

Install the WiX UI extension. You may need to open a new terminal in order to use the newly installed `wix` .NET tool:

```
wix extension add --global WixToolset.UI.wixext/5.0.2
```

> **Important**
>
> To avoid compatibility issues, ensure the WiX UI extension version matches the version of the WiX Toolset.
>
> Run `wix --version` to check the version of WiX Toolset you have installed and replace `5.x.y` with the full version number without the Git revision.

If you want to sign binaries with Authenticode

You'll need a code signing certificate.

To make a .exe

Open a command prompt, cd into the dangerzone directory, and run:

```
poetry run python .\setup-windows.py build
```

In `build\exe.win-amd64-3.14\` you will find `dangerzone.exe`, `dangerzone-cli.exe`, and all supporting files.

To build the installer

Note that you must have a codesigning certificate installed in order to use the `install\windows\build-app.bat` script, because it codesigns `dangerzone.exe`, `dangerzone-cli.exe` and `Dangerzone.msi`.

```
poetry run .\install\windows\build-app.bat
```

When you're done you will have `dist\Dangerzone.msi`.

Using a local container image

It is possible to use a local image for testing, provided you store it under `share/container.tar`. If the local image is not signed, you can bypass signature checks with:

```
export DANGERZONE_BYPASS_SIG_CHECKS=1  # On Linux and macOS
set DANGERZONE_BYPASS_SIG_CHECKS=1  # On Windows
```

To switch back to the original behavior, remove the environment variable:

```
unset DANGERZONE_BYPASS_SIG_CHECKS  # On Linux and macOS
set DANGERZONE_BYPASS_SIG_CHECKS=  # On Windows
```

Dangerzone Testing

Dangerzone has some automated testing under `tests/`.

The following assumes that you have already setup the development environment.

Run tests

Unit / integration tests are run with:

```
poetry run make test
```

Run a subset

The `test` target wraps `pytest`, so you can call it directly with the usual selectors:

```
poetry run pytest tests/test_cli.py
poetry run pytest -k "ocr" -v
```

Lint and type-check

CI runs the same checks as:

```
poetry run make lint
```

Most formatting issues can be fixed automatically with:

```
poetry run make fix
```

Test on another Linux distribution

To run the tests inside a clean container for any supported Debian, Ubuntu, or Fedora release, see Use the containerized dev environments.

Use the containerized dev environments

The `dev_scripts/env.py` script creates environments in which Dangerzone can run. You can also run arbitrary commands in them. In practice, it's useful for generating packages for Linux OSes other than the one installed on your machine. For the design and the caveats behind these environments, see Development environments.

For the full usage information, run `./dev_scripts/env.py --help`.

Pick an environment type

The script supports two types of environments:

- a *dev* environment with the developer tools baked in and the source tree mounted ;
- an *end-user* environment with only Dangerzone installed from a `.deb`/`.rpm` (or from our QA or production repositories with `--qa` / `--prod`).

They are described in Create Dangerzone environments. Each environment corresponds to a Dockerfile, which is generated on the fly. You can see the Dockerfile by passing `--show-dockerfile`.

Build and enter a dev environment

```
./dev_scripts/env.py --distro debian --version bookworm build-dev
./dev_scripts/env.py --distro debian --version bookworm run --dev bash
```

Inside the shell, the source tree is at `/home/user/dangerzone`, so the usual commands work:

```
cd dangerzone
poetry run mazette install
poetry run dangerzone-image prepare-archive --output share/container.tar
poetry run make test
```

Replace `--distro debian --version bookworm` with, for example, `--distro fedora --version 44` or `--distro ubuntu --version noble`.

Run a single command

You can also run one command and get its exit status, which is how the release instructions build packages:

```
./dev_scripts/env.py --distro debian --version bookworm run --dev bash -c \
    "cd dangerzone && ./install/linux/build-deb.py"
```

Build and enter an end-user environment

Build a `.deb` or `.rpm` first (it is picked up from `deb_dist/` or `dist/`), then:

```
./dev_scripts/env.py --distro fedora --version 44 build
./dev_scripts/env.py --distro fedora --version 44 run bash
```

Add `--qa` or `--prod` to `build` to install the package from our QA or production repositories instead, and `--full` to install the `dangerzone-full` variant with the bundled container.

The environments are containers with the X11 socket mounted, so `dangerzone` opens its window on your desktop. Containers are deleted when the command exits, so anything you change inside is lost, on purpose.

Debug the sandbox

When a conversion fails inside the sandbox, or when you work on the container image, it helps to see what happens inside it. This guide shows how to get gVisor logs out of a conversion and how to open a shell in the sandbox image.

Get the sandbox logs for a conversion

Pass `--debug` to the CLI. Dangerzone then starts the container with `RUNSC_DEBUG=1` and prints the gVisor logs together with the conversion output:

```
dangerzone-cli --debug the-document.pdf
```

From a source checkout, `DANGERZONE_DEV=1 poetry run dangerzone-cli --debug ...` does the same.

Open a shell in the sandbox image

The image contains a shell, so you can inspect it as root, outside of gVisor:

Linux

```
podman run --rm -it --user root --entrypoint /bin/sh \
    ghcr.io/freedomofpress/dangerzone/v1
```

macOS and Windows

Use the Podman embedded in Dangerzone, through `dangerzone-machine raw`:

```
dangerzone-machine raw run --rm -it --user root --entrypoint /bin/sh \
    ghcr.io/freedomofpress/dangerzone/v1
```

See the dangerzone-machine reference for the full path of the command.

Inside, the conversion code lives in the `dangerzone.conversion` Python package, and Dangerzone runs a conversion as `/usr/bin/python3 -m dangerzone.conversion.doc_to_pixels`, with the document on standard input. See Sandbox protocol for what it reads and writes.

Note

Running the image by hand like this skips the hardening that Dangerzone applies (no network, dropped capabilities, seccomp profile, gVisor, no user mapping). Only do it with images and documents you trust, or in a disposable environment. You can get a shell under gVisor as well, but you won't get a terminal back, so its usefulness is limited.

Use a locally built image

To test changes to the image itself, build it from the dangerzone-image repository, store the result as `share/container.tar`, and bypass signature checks as described in Use a local container image.

Reset everything

If the local image or the Podman machine is in a broken state:

```
dangerzone-machine reset     # macOS and Windows: remove machine and image
dangerzone-image upgrade     # fetch a fresh, signed image
```

On Linux, remove the image with `podman rmi ghcr.io/freedomofpress/dangerzone/v1` and run `dangerzone-image upgrade`.

Release a new version

The documents in this section are targeted towards Dangerzone release managers who want to publish a new Dangerzone release. If you are looking for a way to run Dangerzone from source, please check the build from source guides instead.

Having a new Dangerzone release out is a rather involved process, because you have to do so simultaneously for all the distributions we support. Based on the experience from past releases, a large list of steps to follow has been assembled.

So, it's either boring uneventful releases or gung-ho brittle ones. Pick your poison (the first one).

This list is broken down in the following phases:

1. Pre-release
2. Prepare build environments
3. Sign and release container image ↗️
4. Build release artifacts
5. QA
6. Release

The checkboxes in these pages can be turned into a tracking issue with `poetry run ./dev_scripts/generate-release-tasks.py`. The release notes templates provide the text for the GitHub release.

Pre-release

Here is a list of tasks that should be done before issuing the release. They can run from the developer's laptop and are not tied to any build environment.

- ☐ Create a new issue named **QA and Release for version \<VERSION>**, to track the general progress. You can generate its content with the `poetry run ./dev_scripts/generate-release-tasks.py` command.
- ☐ Add new Linux platforms and remove obsolete ones
- ☐ Bump the Python dependencies using `poetry lock --regenerate`
- ☐ Bump the GitHub asset dependencies using `poetry run mazette lock`
- ☐ Bump the version of the `dangerzone-image` repo, in case of code changes.
- ☐ Check for new WiX releases and update it if needed (see #1200)
- ☐ Update `version` in `pyproject.toml`
- ☐ Update `share/version.txt`
- ☐ Update `share/version_insecure_converter.txt`, if necessary
- ☐ Update the "Version" field in `install/linux/dangerzone.spec`
- ☐ (optional) Bump image version from `v1` to `v2`, if the API has changed.
- ☐ Bump the Debian version by adding a new changelog entry in `debian/changelog`
- ☐ Run `make docs-cog` to refresh the generated parts of the docs, including the download links in `docs/how-to/install/index.md` (the links will start working once the release assets are published)
- ☐ Update screenshot in `README.md`, if necessary
- ☐ CHANGELOG.md should be updated to include a list of all major changes since the last release
- ☐ A draft release should be created. Copy the release notes text from the release notes templates
- ☐ Send the release notes to editorial for review
- ☐ Tag the tip of the `main` branch as `v<version>-rc1`. E.g. `git tag -s v0.9.1-rc1`.

Add new Linux platforms and remove obsolete ones

Currently supported Linux OSes are Debian, Ubuntu and Fedora (we treat Qubes OS as a special case of Fedora, release-wise). For each of these platforms, check if a new version has been added, or if an existing one is now EOL (https://endoflife.date/ is handy for this purpose).

In case of a new version (beta, RC, or official release):

1. Add it in our CI workflows, to test if that version works.
   - See `.circleci/config.yml` and `.github/workflows/ci.yml`, as well as `dev_scripts/env.py`.
2. Test this version locally, following the QA checklist. Focus on the GUI part, since the basic functionality is already tested by our CI workflows.
3. Add the new version in our `docs/how-to/install/index.md` and `docs/reference/supported-platforms.md` documents, and drop a line in our `CHANGELOG.md`.
4. If that version is a new stable release, update the `docs/how-to/release/` and `docs/how-to/contribute/build-from-source.md` files where necessary.

5. Send a PR with the above changes.

In case of the removal of a version:

1. Remove any mention of this version from our repo.
    - Consult the previous paragraph, but also `grep` your way around.
2. Add a notice in our `CHANGELOG.md` about the version removal.

Prepare build environments

macOS

Initial setup

- Ensure that the build machine has:
  - Apple-trusted `Developer ID Application: Freedom of the Press Foundation (94ZZGGGJ3W)` code-signing certificates installed
- Apple account must have:
  - A valid application password for `notarytool` in the Keychain. You can verify this by running:

    ```
    xcrun notarytool history --apple-id "<email>" --keychain-profile "dz-notarytool-release-key"
    ```

    If the key isn't found, add it to the Keychain by running:

    ```
    xcrun notarytool store-credentials dz-notarytool-release-key --apple-id <email> --team-id 94ZZGGGJ3W
    ```

    with the respective `email`. The team ID is retrieved from https://developer.apple.com -> Account -> Membership details -> Team ID.

On each release

- ☐ Agree to any new terms and conditions in https://developer.apple.com, once you log in with FPF's Apple ID.
- ☐ Upgrade "Command Line Tools" from "System Settings -> Software Update", from an account with admin privileges.
- ☐ Upgrade Xcode from the App Store, from an account with admin privileges.
- ☐ Update Docker Desktop and Podman Desktop to the latest versions.
- ☐ Update Python to the latest supported version, following our instructions

Windows

The Windows release is performed in a Windows 11 virtual machine (as opposed to a physical one).

Initial Setup

- Download a VirtualBox VM image for Windows from here: https://developer.microsoft.com/en-us/windows/downloads/virtual-machines/ and import it into VirtualBox. Also install the Oracle VM VirtualBox Extension Pack.
- Install updates
- Install git for Windows from https://git-scm.com/download/win, and clone the dangerzone repo

- Follow the Windows build instructions, except:
  - Don't install Docker Desktop (it won't work without nested virtualization)
  - Install the Windows SDK from here: https://developer.microsoft.com/en-us/windows/downloads/windows-sdk/ and add `C:\Program Files (x86)\Microsoft SDKs\ClickOnce\SignTool` to the path (you'll need it for `signtool.exe`)
  - You'll also need the Windows codesigning certificate installed on the VM

On each release

- ☐ Update WiX, if necessary
- ☐ Update Python to the latest supported version, following our instructions

Build release artifacts

Follow the sections below, depending on the OS you want to build artifacts for. Linux artifacts can be built on macOS, via the use of containers.

Automated instructions help build things quicker. For an explanation of what they do under the hood, read Build release artifacts with Doit.

macOS

Automated

From your macOS terminal app:

- ☐ `export APPLE_ID=<email>` in your terminal session
- ☐ Build artifacts for Intel macOS with `make build-macos-intel`
- ☐ Build artifacts for Apple Silicon macOS with `make build-macos-arm`

Manual steps

1. Checkout the dependencies, and clean your local copy:

```
# Replace with the actual version
export VERSION=$(cat share/version.txt)

# Verify and checkout the git tag for this release:
git checkout -f v$VERSION

# Clean the git repository
git clean -df

# Clean up the environment
poetry env remove --all

# Install the dependencies
poetry sync
```

2. Retrieve the container image and download the necessary assets:

```
poetry run dangerzone-image prepare-archive \
 --image ghcr.io/freedomofpress/dangerzone/v1@sha256:${DIGEST} \
 --output share/container.tar

poetry run mazette install

# Copy the container image to the assets folder
cp share/container.tar ~dz/release-assets/$VERSION/dangerzone-$VERSION-arm64.tar
```

3. Build the app bundle

```
poetry run ./install/macos/build-app.py
```

4. Sign the application bundle, and notarize it

   This command needs to run from an account with access to the code-signing certificate.

   This command assumes the Apple Developer ID application password is stored in the Keychain and used specifically for `notarytool`.

```
# Sign the .App and make it a .dmg
poetry run ./install/macos/build-app.py --only-codesign

# Notarize it. This command must run from the MacOS UI,
# inside a terminal application.
xcrun notarytool submit ./dist/Dangerzone.dmg --apple-id $APPLE_ID --keychain-profile "dz-
notarytool-release-key" --wait && xcrun stapler staple dist/Dangerzone.dmg

# Copy the .dmg to the assets folder
ARCH=$(uname -m)
if [ "$ARCH" = "x86_64" ]; then
    ARCH="i686"
fi
cp dist/Dangerzone.dmg ~dz/release-assets/$VERSION/Dangerzone-$VERSION-$ARCH.dmg
```

Windows

- ☐ Checkout the dependencies, and clean the local copy:

```
# Replace with the actual version
export VERSION=$(cat share/version.txt)

# Verify and checkout the git tag for this release:
git checkout -f v$VERSION

# Clean the git repository
git clean -df

# Clean up the environment
poetry env remove --all

# Install the dependencies
poetry sync
```

- ☐ Download the container image with signatures:

```
poetry run dangerzone-image prepare-archive --image ghcr.io/freedomofpress/dangerzone/
v1@sha256:${DIGEST} --output share/container.tar
```

- ☐ Download the necessary assets with `poetry run mazette install`
- ☐ Run `poetry run .\install\windows\build-app.bat`. After completion, the installer will be available at `dist\Dangerzone.msi`
- ☐ Rename `Dangerzone.msi` to `Dangerzone-$VERSION.msi`.

Linux

Automated

- ☐ Run `make build-linux` (not necessary if you've previously built artifacts for macOS)

Manual steps

Below we explain how we build packages for each Linux distribution we support.

Debian/Ubuntu

Because the Debian packages do not contain compiled Python code for a specific Python version, a single Debian package can be used for all of our Debian-based distros.

Create a Debian Bookworm development environment. Follow the instructions in the build section, or create it locally with:

```
# Create and run debian bookworm development environment
./dev_scripts/env.py --distro debian --version bookworm build-dev
./dev_scripts/env.py --distro debian --version bookworm run --dev bash

# Get the vendorized assets
poetry run mazette install

# Retrieve the latest container
poetry run dangerzone-image prepare-archive \
    --image ghcr.io/freedomofpress/dangerzone/v1@sha256:${DIGEST} \
    --output share/container.tar

# Create both .deb packages (dangerzone slim + dangerzone-full)
./dev_scripts/env.py --distro debian --version bookworm run --dev bash -c "cd dangerzone && ./
install/linux/build-deb.py"
```

A single `build-deb.py` invocation produces both `dangerzone_<version>_amd64.deb` (slim, no bundled `container.tar`) and `dangerzone-full_<version>_amd64.deb` (with the container image bundled) under `./deb_dist`.

Publish both `.deb` files under `./deb_dist` to the `freedomofpress/packages` repo, by sending a PR. Follow the instructions in that repo on how to do so.

Fedora

> **Note**
>
> This procedure will have to be done for every supported Fedora version. In this section, Fedora 42 is used as an example.

Create a Fedora development environment. Follow the instructions in the build section, or create it locally with:

```
./dev_scripts/env.py --distro fedora --version 42 build-dev
./dev_scripts/env.py --distro fedora --version 42 run --dev bash
```

```
# Get the vendorized assets
poetry run mazette install

# Retrieve the latest container
poetry run dangerzone-image prepare-archive \
    --image ghcr.io/freedomofpress/dangerzone/v1@sha256:${DIGEST} \
    --output share/container.tar

# Create the dangerzone .rpm (without container):
./dev_scripts/env.py --distro fedora --version 42 run --dev bash -c "cd dangerzone && ./install/
linux/build-rpm.py"
# Create the dangerzone-full .rpm (with container bundled):
./dev_scripts/env.py --distro fedora --version 42 run --dev bash -c "cd dangerzone && ./install/
linux/build-rpm.py --full"
```

Publish both `dangerzone-<version>.fc<NN>.x86_64.rpm` (slim) and `dangerzone-full-<version>.fc<NN>.x86_64.rpm` (bundled container) under `./dist` to the freedomofpress/packages repo, by sending a PR. Follow the instructions in that repo on how to do so.

Qubes

Create a `.rpm` for Qubes:

```
./dev_scripts/env.py --distro fedora --version 42 run --dev bash -c "cd dangerzone && ./install/
linux/build-rpm.py --qubes"
```

and similarly publish it to the freedomofpress/packages repo.

QA

The produced packages are tested to make sure that new releases do not introduce regressions, and that they support existing and newer platforms.

Here is a list of checks. Some of them are manual, and the release manager needs to follow them across several OSes.

There is also a large collection of documents that can be tested against the `main` branch prior to a release (see Large Document Testing).

The checklist

- ☐ Make sure that the tip of the `main` branch passes the CI tests.
- ☐ Create a test build in Windows and make sure it works:
    - ☐ Check if the suggested Python version is still supported.
    - ☐ Create a new development environment with Poetry.
    - ☐ Build the container image and ensure the development environment uses the new image.
    - ☐ Download the necessary assets using `poetry run mazette install`
    - ☐ Run the Dangerzone tests.
    - ☐ Build and run the Dangerzone .exe
    - ☐ Test some QA scenarios (see Scenarios below).
- ☐ Create a test build in macOS (Intel CPU) and make sure it works:
    - ☐ Check if the suggested Python version is still supported.
    - ☐ Create a new development environment with Poetry.
    - ☐ Build the container image and ensure the development environment uses the new image.
    - ☐ Download the necessary assets using `poetry run mazette install`
    - ☐ Run the Dangerzone tests.
    - ☐ Create and run an app bundle.
    - ☐ Test some QA scenarios (see Scenarios below).
- ☐ Create a test build in macOS (M½ CPU) and make sure it works:
    - ☐ Check if the suggested Python version is still supported.
    - ☐ Create a new development environment with Poetry.
    - ☐ Build the container image and ensure the development environment uses the new image.
    - ☐ Download the necessary assets using `poetry run mazette install`
    - ☐ Run the Dangerzone tests.
    - ☐ Create and run an app bundle.
    - ☐ Test some QA scenarios (see Scenarios below).
- ☐ Create a test build in the most recent Ubuntu LTS platform (Ubuntu 24.04 as of writing this) and make sure it works:
    - ☐ Create a new development environment with Poetry.
    - ☐ Build the container image and ensure the development environment uses the new image.
    - ☐ Download the necessary assets using `poetry run mazette install`

        - ☐ Run the Dangerzone tests.
        - ☐ Create a .deb package and install it system-wide.
        - ☐ Test some QA scenarios (see Scenarios below).
- ☐ Create a test build in the most recent Fedora platform (Fedora 44 as of writing this) and make sure it works:
    - ☐ Create a new development environment with Poetry.
    - ☐ Build the container image and ensure the development environment uses the new image.
    - ☐ Download the necessary assets using `poetry run mazette install`
    - ☐ Run the Dangerzone tests.
    - ☐ Create an .rpm package and install it system-wide.
    - ☐ Test some QA scenarios (see Scenarios below).
- ☐ Create a test build in the most recent Qubes Fedora template (Fedora 44 as of writing this) and make sure it works:
    - ☐ Create a new development environment with Poetry.
    - ☐ Run the Dangerzone tests.
    - ☐ Create a Qubes .rpm package and install it system-wide.
    - ☐ Ensure that the Dangerzone application appears in the "Applications" tab.
    - ☐ Test some QA scenarios (see Scenarios below) and make sure they spawn disposable qubes.

Scenarios

1. Updating Dangerzone handles external state correctly.

(Applies to Windows/MacOS)

Install the previous version of Dangerzone, downloaded from the website.

Open the Dangerzone application and enable some non-default settings. **If there are new settings, make sure to change those as well**.

Close the Dangerzone application and get the container image for that version. For example:

```
$ dangerzone-machine raw images ghcr.io/freedomofpress/dangerzone/v1
REPOSITORY                          TAG         IMAGE ID      CREATED       SIZE
ghcr.io/freedomofpress/dangerzone/v1  <tag>       <image ID>    <date>        <size>
```

Then run the version under QA and ensure that the settings remain changed.

Afterwards, check that a new docker image was installed by running the same command and looking for the following differences:

```
$ dangerzone-machine raw images ghcr.io/freedomofpress/dangerzone/v1
REPOSITORY                          TAG         IMAGE ID        CREATED       SIZE
ghcr.io/freedomofpress/dangerzone/v1  <other tag> <different ID>  <newer date>  <different size>
```

2. Dangerzone successfully installs the container image

(Only for Linux)

Remove the Dangerzone container image and podman machine with:

```
dangerzone-machine reset
```

Then run Dangerzone. Dangerzone should install the podman machine and container image successfully.

3. Dangerzone retains the settings of previous runs

Run Dangerzone and make some changes in the settings (e.g., change the OCR language, toggle whether to open the document after conversion, etc.). Restart Dangerzone. Dangerzone should show the settings that the user chose.

4. Dangerzone reports failed conversions

Run Dangerzone and convert the `tests/test_docs/sample_bad_pdf.pdf` document. Dangerzone should fail gracefully, by reporting that the operation failed, and showing the following error message:

> The document format is not supported

5. Dangerzone succeeds in converting multiple documents

Run Dangerzone against a list of documents, and tick all options. Ensure that:

- Conversions take place sequentially.
- Attempting to close the window while converting asks the user if they want to abort the conversions.
- Conversions are completed successfully.
- Conversions show individual progress in real-time (double-check for Qubes).
- *(Only for Linux)* The resulting files open with the PDF viewer of our choice.
- OCR seems to have detected characters in the PDF files.
- The resulting files have been saved with the proper suffix, in the proper location.
- The original files have been saved in the `unsafe/` directory.

6. Dangerzone is able to handle drag-n-drop

Run Dangerzone against a set of documents that you drag-n-drop. Files should be added and conversion should run without issue.

> **Tip**
>
> On our end-user container environments for Linux, we can start a file manager with `thunar &`.

7. Dangerzone CLI succeeds in converting multiple documents

(Only for Windows and Linux)

Run Dangerzone CLI against a list of documents. Ensure that conversions happen sequentially, are completed successfully, and we see their progress.

8. Dangerzone can open a document for conversion via right-click -> "Open With"

(Only for Windows, MacOS and Qubes)

Go to a directory with office documents, right-click on one, and click on "Open With". We should be able to open the file with Dangerzone, and then convert it.

9. Dangerzone shows helpful errors for setup issues on Qubes

(Only for Qubes)

Check what errors Dangerzone throws in the following scenarios. The errors should point the user to the Qubes notifications in the top-right corner:

1. The `dz-dvm` template does not exist. We can trigger this scenario by temporarily renaming this template.
2. The Dangerzone RPC policy does not exist. We can trigger this scenario by temporarily renaming the `dz.Convert` policy.
3. The `dz-dvm` disposable Qube cannot start due to insufficient resources. We can trigger this scenario by temporarily increasing the minimum required RAM of the `dz-dvm` template to more than the available amount.

Large Document Testing

Parallel to the QA process, the release candidate should be put through the large document tests. This can be done by rebasing the currently open Pull Request.

These tests will identify any regressions or improvements in document coverage.

Release

Once you are confident that the release doesn't need any more changes:

- ☐ Create a PGP-signed git tag for the version, e.g., for dangerzone `v0.1.0`:

  ```
  git tag -s v0.1.0
  git push origin v0.1.0
  ```

  Pushing the tag publishes the documentation for this version on docs.dangerzone.rocks and points the `latest` alias (the site default) to it. Between releases, pushes to `main` only update the `dev` version of the site.

- ☐ Create an archive of the Dangerzone source in `tar.gz` format:

  ```
  export VERSION=$(cat share/version.txt)
  git archive --format=tar.gz -o dangerzone-${VERSION:?}.tar.gz --prefix=dangerzone/ v${VERS
  ION:?}
  ```

  (This is covered by our automated build steps)

- ☐ Ensure that https://cves.dangerzone.rocks does not report any serious vulnerabilities.

- ☐ Collect the assets in a single directory, calculate their SHA-256 hashes, and sign them. There is an `./dev_scripts/sign-assets.py` script to automate this task.

  ```
  # Sign all the assets
  ./dev_scripts/sign-assets.py ~/release-assets/$VERSION/github --version $VERSION
  ```

- ☐ Upload all the assets to the draft release on GitHub.

  ```
  find ~/release-assets/$VERSION/github | xargs -n1 ./dev_scripts/upload-asset.py --token ~/
  token --draft
  ```

- ☐ Update the draft release to target the final git tag.

- ☐ Send a PR to update the Dangerzone website with the new release announcement.

📣 Publish the release!

To actually publish the release:

- ☐ Merge the PR(s) in the `packages` repository.
- ☐ Make the GitHub draft release public.
- ☐ Merge the PRs in `dangerzone.rocks` and `dangerzone`.
- ☐ Toot release announcement on our mastodon account https://social.freedom.press/@dangerzone
- ☐ Extend the `check_repos.yml` CI test for the newly added platforms, if necessary
- ☐ Manually trigger the `check_repos.yml` CI test and ensure it passes.

Release notes templates

Copy one of these templates into the body of the draft GitHub release, then fill in the placeholders. The files live under `dev_scripts/templates/` in the repository.

Regular release

`dev_scripts/templates/release-notes-regular.md`:

```
This release includes various new features, stability improvements, and security fixes **(adjust
accordingly)**.

The highlights for this release are:
  - **Important accomplishment**
    We used to do [this](https://github.com/freedomofpress/dangerzone/issues/1), but now we do
[that](https://github.com/freedomofpress/dangerzone/issues/2).
  - **Support for a new platform**
    We added support for a new platform ([#3](https://github.com/freedomofpress/dangerzone/issues/
3))
  - **Community contributions**
     <!-- Acknowledge all contributions and talk about highlights ->

For a full list of the changes, see our [changelog](https://github.com/freedomofpress/dangerzone/
blob/<RELEASE_TAG>/CHANGELOG.md#<RELEASE_ANCHOR>).
```

Security release

`dev_scripts/templates/release-notes-security.md`:

```
This is a security release that mainly addresses CVE-XXXX-XXX. Our [security advisory](https://
github.com/freedomofpress/dangerzone/blob/<RELEASE_TAG>/docs/advisories/<YYYY-MM-DD>.md) follows:

<!-- Vulnerability description for non-technical users -->

**To reduce that risk, you are strongly advised to update Dangerzone to the latest version**.

# Summary

# How does this impact me?

# What do I need to do?

You are **strongly** advised to update your Dangerzone installation to <VERSION> as soon as
possible.


---

On other news, this release brings a fix for ([#4](https://github.com/freedomofpress/dangerzone/
issues/4)) and a fix for ([#5](https://github.com/freedomofpress/dangerzone/issues/5))

For a full list of the changes, see our [changelog](https://github.com/freedomofpress/dangerzone/
blob/<RELEASE_TAG>/CHANGELOG.md#<RELEASE_ANCHOR>).
```

The security template mirrors the structure of our security advisories.

Managing Python installations

macOS

- ☐ Verify and install the latest supported Python version from python.org

  > 🚨 Do not use the one from Homebrew as it is known to cause issues

- ☐ Ensure the `~/.zprofile` of all accounts contains the following lines, and remove any older ones, if they exist:

```
PATH="/Library/Frameworks/Python.framework/Versions/<version>/bin:${PATH}"
export PATH
```

(make sure to replace `<version>` with the version of the newly installed Python, e.g., `3.14`)

> 🚨 When a user installs Python from python.org, it is installed by default in a non-standard location (`/Library/Frameworks/Python.framework/Versions/`), and can co-exist with previous installations from https://python.org. In order to make it accessible to the shell, the Python installer updates the `PATH` environment variable. However, it does so only for the account that installs Python, and not for the rest of the accounts. So, the last part must be done manually.

- ☐ Exit any previous terminal/SSH sessions, so that the `$PATH` changes can take effect.

(We can't just `source ~/.zshrc`, because it doesn't account for removals or changes in `~/.zprofile`)

- ☐ Ensure `which python3` returns the expected Python installation

```
% which python3
/Library/Frameworks/Python.framework/Versions/3.14/bin/python3
```

- ☐ Install Poetry from an account with **admin** privileges, with `python3 -m pip install -U poetry`

- ☐ Ensure `which poetry` returns the expected Poetry installation

```
% which poetry
/Library/Frameworks/Python.framework/Versions/3.14/bin/poetry
```

- ☐ Ensure that Poetry picks the correct Python version with `poetry debug info`

```
% cd dangerzone
% poetry debug info
Poetry
Version: 2.1.3
Python:  3.14.8
```

```
Virtualenv
Python:         3.14.8
Implementation: CPython
Path:           [...]
Executable:     [...]
Valid:          True

Base
Platform:   darwin
OS:         posix
Python:     3.14.8
Path:       [...]
Executable: [...]
```

Windows

- ☐ Verify and install the latest supported Python version from python.org
- ☐ Install Poetry with `python -m pip install -U poetry`
- ☐ Ensure that Poetry picks the correct Python version with `poetry debug info`

```
C:\> cd dangerzone
C:\> poetry debug info
Poetry
Version: 2.5.1
Python:  3.14.8

Virtualenv
Python:         3.14.8
Implementation: CPython
Path:           [...]
Executable:     [...]
Valid:          True

Base
Platform:   win32
OS:         nt
Python:     3.14.8
Path:       [...]
Executable: [...]
```

Using the Doit Automation Tool

Developers can use the Doit automation tool to create release artifacts. The purpose of the tool is to automate the manual release instructions. Not everything is automated yet, since we're still experimenting with this tool. You can find our task definitions in this repo's `dodo.py` file.

Why Doit?

We picked Doit out of the various tools out there for the following reasons:

- **Pythonic:** The configuration file and tasks can be written in Python. Where applicable, it's easy to issue shell commands as well.
- **File targets:** Doit borrows the file target concept from Makefiles. Tasks can have file dependencies, and targets they build. This makes it easy to define a dependency graph for tasks.
- **Hash-based caching:** Unlike Makefiles, doit does not look at the modification timestamps of source/target files to figure out if it needs to run a task. Instead, it hashes those files, and will run a task only if the hash of a file dependency has changed.
- **Parallelization:** Tasks can be run in parallel with the `-n` argument, which is similar to `make`'s `-j` argument.

How to Doit?

First, enter your Poetry shell. Then, make sure that your environment is clean, and you have ample disk space. You can run:

```
doit clean --dry-run  # if you want to see what would happen
doit clean  # you'll be asked to confirm that you want to clean everything
```

Finally, you can build all the release artifacts with `doit`, or a specific task with:

```
doit <task>
```

Tips and tricks

- You can run `doit list --all -s` to see the full list of tasks, their dependencies, and whether they are up to date (U) or will run (R). Note that certain small tasks are always configured to run.
- You can run `doit info <task>` to see which dependencies are missing.
- You can pass the following environment variables to the script, in order to affect some global parameters:
    - `CONTAINER_RUNTIME`: The container runtime to use. Either `podman` (default) or `docker`.
    - `RELEASE_DIR`: Where to store the release artifacts. Default path is `~/release-assets/<version>`
    - `APPLE_ID`: The Apple ID to use when signing/notarizing the macOS DMG.

Reference

Supported document formats

Dangerzone can convert these types of document into safe PDFs:

- PDF (`.pdf`)
- Microsoft Word (`.docx`, `.doc`)
- Microsoft Excel (`.xlsx`, `.xls`)
- Microsoft PowerPoint (`.pptx`, `.ppt`)
- ODF Text (`.odt`)
- ODF Spreadsheet (`.ods`)
- ODF Presentation (`.odp`)
- ODF Graphics (`.odg`)
- Hancom HWP (Hangul Word Processor) (`.hwp`, `.hwpx`)
    - Not supported on Qubes OS
- EPUB (`.epub`)
- Jpeg (`.jpg`, `.jpeg`)
- GIF (`.gif`)
- PNG (`.png`)
- SVG (`.svg`)
- other image formats (`.bmp`, `.pnm`, `.pbm`, `.ppm`, `.tif`, `.tiff`)

The output is always a PDF. Its name is the input name with a `-safe.pdf` suffix, unless you choose otherwise. See How Dangerzone works.

OCR languages

OCR is performed with Tesseract. The graphical application lists the languages by name. The command-line tool takes Tesseract language codes with `--ocr-lang`, for example `eng`, `fra`, `deu`, `spa`, `ara`, `chi_sim`, or `jpn`. Passing an unknown code prints the full list of accepted codes. The mapping between names and codes is in `share/ocr-languages.json` in the source tree.

Operating System support

Dangerzone can run on various Operating Systems (OS), and has automated tests for most of them. This section explains which OS we support, how long we support each version, and how we test Dangerzone against these.

You can find general support information in this table, and more details in the following sections.

(Unless specified, the architecture of the OS is AMD64)

Distribution	Supported releases	Automated tests	Manual QA
Windows	2 last releases ⚓	(`windows-2022`, `windows-2025`) ◎	
macOS intel	3 last releases	(`macos-15`) ◎	
macOS silicon	3 last releases	(`macos-15`) ◎	
Ubuntu	Follow upstream support		
Debian	Current stable, Oldstable and LTS releases		
Fedora	Follow upstream support		
Qubes OS	Beta support ✢		Latest Fedora template
Tails	Only the last release		Last release only

Notes:

⚓ We provide support for Windows 10 on a best effort basis, given that it is currently end of life. Windows on Arm is not supported at this time.

✢ Qubes OS support assumes the use of a Fedora template. The supported releases follow our general support for Fedora.

◎ More information in the runner-images repository

Note on unsupported Linux distros

`.deb` and `.rpm` packages are provided for supported distributions. Users of **other** Debian-based or Fedora-based distros that are not listed above may be able to install Dangerzone through these packages. Unfortunately, Dangerzone is not tested against these distros, and might fail to install, update, run, or be broken in subtle ways.

Please, proceed at your own risk, only if you know what you're doing.

dangerzone-cli

`dangerzone-cli` converts one or more documents into safe PDFs from a terminal. It uses the same conversion pipeline and the same sandbox as the graphical application. For a guided introduction, see Convert documents from the command line.

Location

Platform	Command
Linux	`dangerzone-cli`
macOS	`/Applications/Dangerzone.app/Contents/MacOS/dangerzone-cli`
Windows	`C:\Program Files\Dangerzone\dangerzone-cli.exe`
Source tree	`poetry run dangerzone-cli`

Usage

The output below is generated from the code at the time the documentation was built, so it matches the installed version of the same release.

```
Usage: dangerzone-cli [OPTIONS] [FILENAMES]...

  Convert potentially dangerous documents to safe PDFs.

  Documents are converted inside a sandbox, so that any embedded threats are
  neutralized. You can also use OCR to add a searchable text layer to the safe
  PDF, via the --ocr-lang option.

  Pass one or more file paths as arguments, or use '-' to read a document from
  standard input. For single-document conversions, you can write the safe PDF
  to a file with '-o <file>', or to standard output with '-o -'.

  Examples:

      dangerzone-cli evidence.odt                 # convert to 'evidence-safe.pdf'
      dangerzone-cli doc1.pdf doc2.docx           # convert to 'doc1-safe.pdf' and 'doc2-safe.pdf'
      dangerzone-cli --archive report.docx        # convert to 'report-safe.pdf' and move the
original under './unsafe/'
      dangerzone-cli --ocr-lang eng scan.pdf      # add a searchable text layer in English
      dangerzone-cli -o out.pdf in.pdf            # write to 'out.pdf'
      dangerzone-cli -o - in.pdf > out.pdf        # write to stdout and pipe to 'out.pdf'
      dangerzone-cli - -o out.pdf < in.pdf        # read from stdin and write to 'out.pdf'
      dangerzone-cli - -o - < in.pdf > out.pdf    # read from stdin, write to stdout and pipe to
'out.pdf'

Options:
  -o, --output-filename TEXT    Output filename for the safe PDF. Default is
                                the input filename with '-safe.pdf' appended.
                                Alternatively, use '-' to write the safe PDF
                                to standard output.
  --ocr-lang TEXT               Language to OCR, defaults to none
  --archive                     Archives the unsafe version in a subdirectory
                                named 'unsafe'
  --debug                       Run Dangerzone in debug mode, to get logs from
```

```
                              gVisor.
  --set-container-runtime TEXT  The name or full path of the container runtime
                              you want Dangerzone to use. You can specify
                              the value 'default' if you want to take back
                              your choice, and let Dangerzone use the
                              default runtime for this OS
  --linger                    Do not stop the Podman machine VM that
                              Dangerzone uses to run containers, after the
                              conversions have completed. This is useful if
                              you want to run multiple conversions in a row,
                              since the startup of the VM takes some time.
                              If you choose to let the Podman machine
                              linger, you will need to stop it manually with
                              `dangerzone-machine stop`. This option affects
                              only Windows/macOS platforms.
  --version                   Show the version and exit.
  --help                      Show this message and exit.
```

`FILENAMES...` are the documents to convert. Every file must exist and must have one of the supported formats. Conversions run sequentially, in the given order. `--output-filename` is only valid with a single input file, and the name must end with `.pdf`. `--set-container-runtime` stores the choice in the settings file and exits without converting anything, see Using Podman Desktop.

Behaviour

On start, the tool prints a banner and then, when needed, installs the Windows Subsystem for Linux, stops other Podman machines, initializes and starts the Dangerzone Podman machine, checks for updates, and installs the sandbox image. On Linux, none of the machine steps apply. On Qubes OS with the native integration, conversions run in disposable qubes.

> **Slim Linux packages**
>
> On Linux, the `dangerzone` package doesn't come with the sandbox bundled, and so the CLI does not download the sandbox on its own.
>
> Initialize it with `dangerzone-image upgrade`, or start the graphical application and accept the download. Alternatively, the `dangerzone-full` package bundles the sandbox.

Progress for each stage of each document is printed to the terminal. A failed conversion reports the error and moves on to the next document.

Exit status

`0`
: All conversions succeeded (or the runtime was set with `--set-container-runtime`).

`1`
: A usage error, an invalid OCR language, or at least one failed conversion.

Environment

`DANGERZONE_DEV`, `DANGERZONE_BYPASS_SIG_CHECKS`, `QUBES_CONVERSION` and the other variables listed in environment variables affect this tool.

Examples

Convert a single PDF next to itself:

```
$ dangerzone-cli report.pdf
```

Convert several office documents with English OCR and archive the originals:

```
$ dangerzone-cli --ocr-lang eng --archive *.docx *.pptx
```

Choose the output name:

```
$ dangerzone-cli --output-filename ~/clean/report.pdf report.pdf
```

Point Dangerzone at Podman Desktop on macOS, then revert:

```
$ /Applications/Dangerzone.app/Contents/MacOS/dangerzone-cli --set-container-runtime /opt/podman/
bin/podman
$ /Applications/Dangerzone.app/Contents/MacOS/dangerzone-cli --set-container-runtime default
```

dangerzone-image

`dangerzone-image` manages the sandbox container image that Dangerzone uses to convert documents: it fetches updates, verifies signatures, and prepares or loads archives for air-gapped machines. For step-by-step usage, see Independent Container Updates. For the design, see Independent sandbox updates.

Location

Platform	Command
Linux	`dangerzone-image`
macOS	`/Applications/Dangerzone.app/Contents/MacOS/dangerzone-image`
Windows	`C:\Program Files\Dangerzone\dangerzone-image.exe`
Source tree	`poetry run dangerzone-image`

Usage

The output below is generated from the code at the time the documentation was built, so it matches the installed version of the same release.

```
Usage: dangerzone-image [OPTIONS] COMMAND [ARGS]...

Options:
  --debug
  --help   Show this message and exit.

Commands:
  load-archive      Use ARCHIVE_FILENAME as the dangerzone sandbox image
  prepare-archive   Prepare an archive to upgrade the dangerzone image...
  store-signatures  Retrieves and stores the signatures of the remote...
  upgrade           Upgrade the sandbox to the latest version available.
  verify-local      Ensures local image signature(s) match the embedded...
```

Commands

`upgrade`

```
Usage: dangerzone-image upgrade [OPTIONS]

  Upgrade the sandbox to the latest version available.

  To upgrade to a custom sandbox image, use "prepare-archive" and "load-
  archive" instead.

Options:
  --help  Show this message and exit.
```

The tool resolves the digest of the default image in the registry, downloads it, verifies its Cosign signatures against the bundled public key, and stores the signatures locally. If the local image is already up to date, it says so and exits successfully. Requires a working container runtime.

`verify-local`

```
Usage: dangerzone-image verify-local [OPTIONS]

  Ensures local image signature(s) match the embedded public key.

Options:
  --image TEXT  The name of the image to check signatures for  [default:
                ghcr.io/freedomofpress/dangerzone/v1]
  --help        Show this message and exit.
```

Verifies the image that is installed locally. Requires a working container runtime.

`prepare-archive`

```
Usage: dangerzone-image prepare-archive [OPTIONS]

  Prepare an archive to upgrade the dangerzone image (useful for airgapped
  environment)

Options:
  --image TEXT   The sandbox container registry location  [default:
                 ghcr.io/freedomofpress/dangerzone/v1]
  --output TEXT  The location of the generated archive. '{arch}' will be
                 replaced by the specified or detected architecture (see
                 --arch)  [default: dangerzone-{arch}.tar]
  --arch TEXT    The architecture to prepare the archive for.  [default: (the
                 architecture of the current machine)]
  --help         Show this message and exit.
```

Download the target container image as a `.tar` archive, and prepare it so that it can be loaded by Dangerzone. The archive contains the image, its signatures, and a `dangerzone.json` manifest, so that the receiving machine can verify it without network access. A digest can be pinned with `--image name@sha256:...`. `--arch` accepts `amd64` or `arm64`. This command does not need a container runtime.

`load-archive`

```
Usage: dangerzone-image load-archive [OPTIONS] ARCHIVE_FILENAME

  Use ARCHIVE_FILENAME as the dangerzone sandbox image

Options:
  --force  Force the installation, bypassing logindex verification checks
  --help   Show this message and exit.
```

Install a `.tar` archive. The signatures in the archive are verified before the image is loaded, and the tool refuses to install an image older than the currently installed one unless `--force` is given. Requires a working container runtime.

`store-signatures`

```
Usage: dangerzone-image store-signatures [OPTIONS]

  Retrieves and stores the signatures of the remote sandbox

Options:
  --image TEXT  [default: ghcr.io/freedomofpress/dangerzone/v1]
  --help        Show this message and exit.
```

Fetches and stores the signatures without pulling the image itself.

Exit status

Commands print a ✅ line on success and exit with `0`. On failure they print a ❌ line and abort with a non-zero status. Typical failures are signature verification errors and an archive older than the installed image.

Environment

`SIGSTORE_REKOR_PUBLIC_KEY` overrides the bundled Rekor public key, and `DANGERZONE_BYPASS_SIG_CHECKS` disables signature verification for local testing. See environment variables.

Examples

Fetch the latest sandbox:

```
$ dangerzone-image upgrade
✅ The local image ghcr.io/freedomofpress/dangerzone/v1 has been upgraded
✅ The image has been signed with /usr/share/dangerzone/freedomofpress-dangerzone.pub
✅ Signatures have been verified and stored locally
```

Prepare an archive for an air-gapped `amd64` machine, then load it there:

```
$ dangerzone-image prepare-archive --arch amd64
✅ Archive dangerzone-amd64.tar created
$ dangerzone-image load-archive dangerzone-amd64.tar
✅ Installed image dangerzone-amd64.tar on the system as ghcr.io/freedomofpress/dangerzone/v1 with
digest sha256:…
```

dangerzone-machine

`dangerzone-machine` manages the Podman machine that Dangerzone uses on macOS and Windows. A Podman machine is a small Linux virtual machine in which the sandbox containers run. Dangerzone creates and starts its own machine automatically, so this tool is mostly useful for troubleshooting or resetting a broken installation.

On Linux, Podman runs natively and no virtual machine is involved.

Location

Platform	Command
macOS	`/Applications/Dangerzone.app/Contents/MacOS/dangerzone-machine`
Windows	`C:\Program Files\Dangerzone\dangerzone-machine.exe`
Source tree	`poetry run dangerzone-machine`

Usage

The output below is generated from the code at the time the documentation was built, so it matches the installed version of the same release.

```
Usage: dangerzone-machine [OPTIONS] COMMAND [ARGS]...

  Manage Dangerzone Podman machines.

Options:
  --log-level [debug|info|warning|error|critical]
                                  Set the logging level.
  --help                          Show this message and exit.

Commands:
  init    Initialize a Dangerzone Podman machine.
  list    List Dangerzone Podman machines.
  raw     Run a raw Podman command.
  remove  Remove the Dangerzone Podman machine.
  reset   Reset all Podman machines.
  start   Start the Dangerzone Podman machine.
  stop    Stop the Dangerzone Podman machine.
```

Commands

`init`

```
Usage: dangerzone-machine init [OPTIONS]

  Initialize a Dangerzone Podman machine.

Options:
  --cpus INTEGER    Number of CPUs to allocate.
  --memory INTEGER  Amount of memory in bytes.
```

```
  --timezone TEXT   Timezone for the machine.
  --help            Show this message and exit.
```

`list`

```
Usage: dangerzone-machine list [OPTIONS]

  List Dangerzone Podman machines.

Options:
  --help  Show this message and exit.
```

`raw`

```
Usage: dangerzone-machine raw [OPTIONS]

  Run a raw Podman command.

Options:
  --help  Show this message and exit.
```

`remove`

```
Usage: dangerzone-machine remove [OPTIONS]

  Remove the Dangerzone Podman machine.

Options:
  -f, --force  Force removal without prompt.
  --help       Show this message and exit.
```

`reset`

```
Usage: dangerzone-machine reset [OPTIONS]

  Reset all Podman machines.

Options:
  -f, --force  Force reset without prompt.
  --help       Show this message and exit.
```

`start`

```
Usage: dangerzone-machine start [OPTIONS]

  Start the Dangerzone Podman machine.

Options:
  --help  Show this message and exit.
```

`stop`

```
Usage: dangerzone-machine stop [OPTIONS]

  Stop the Dangerzone Podman machine.

Options:
  --help  Show this message and exit.
```

Notes on individual commands:

- `init`, `start`, and `stop` require the Windows Subsystem for Linux on Windows. Use `stop` after running `dangerzone-cli --linger`.
- `reset` runs `podman machine reset`. This is a destructive action: it removes every Podman machine on the system, including the Dangerzone one and the sandbox image stored in it, so Dangerzone installs both again on next start.
- `raw` runs a Podman command with the Podman binary and connection that Dangerzone uses. Everything after `raw` is passed to Podman untouched.

Exit status

`0` on success. On a Podman error, the tool prints a ❌ line and aborts with a non-zero status. Declining a confirmation prompt also aborts.

Examples

Check whether the machine is running:

```
$ dangerzone-machine list
Name: dz-internal-0.11.0, Status: Running
```

Inspect the sandbox images known to the machine:

```
$ dangerzone-machine raw images ghcr.io/freedomofpress/dangerzone/v1
```

Start over from a clean state:

```
$ dangerzone-machine reset
Are you sure you want to reset all Podman machines? This is a destructive action. [y/N]: y
Podman machines reset.
```

Sandbox protocol

This page describes the interface between Dangerzone (the host) and the sandbox that converts a document to pixels. It is the "API" of the container image. The image and the conversion code it runs are maintained in the dangerzone-image repository. The host side lives in `dangerzone/isolation_provider/`.

Invocation

For each document, Dangerzone starts one container from the sandbox image and runs:

```
/usr/bin/python3 -m dangerzone.conversion.doc_to_pixels
```

On Qubes OS with the native integration, the equivalent is the `dz.Convert` RPC call to a disposable qube (and `dz.ConvertDev` during development).

The container is started with these flags, in addition to the image digest pinned after signature verification:

Flag	Purpose
`--network=none`	No network access
`--cap-drop all --cap-add SYS_CHROOT`	Drop every capability except the one gVisor needs to set up its root
`--security-opt no-new-privileges`	No privilege escalation
`--security-opt seccomp=<profile>`	Bundled seccomp profile that allows gVisor to run (`share/seccomp.gvisor.json`), since default profiles may block `ptrace(2)`
`--userns nomap`	Do not map the host user into the container (Podman 4.1 and later)
`--security-opt label=type:container_engine_t`	SELinux label that lets a container engine run nested, on hosts that use SELinux
`-u dangerzone`	Run as the unprivileged `dangerzone` user inside the container
`--log-driver none`	Do not keep container logs
`--rm`	Remove the container when it exits
`-i`	Keep standard input open
`-e RUNSC_DEBUG=1`	Only with `--debug`: make gVisor log verbosely

Inside the container, the conversion runs under gVisor (`runsc`), which in turn spawns an inner container, with no capabilities and privileges. You can read more about it in the gVisor integration docs.

Input

The host writes the raw bytes of the untrusted document to the container's standard input, then closes it. No filename, format, or options are sent: the sandbox detects the format itself.

Output

The container writes a binary stream to standard output. All integers are 2-byte unsigned big-endian values (`INT_BYTES = 2`).

```
n_pages                      uint16
repeat n_pages times:
    width                    uint16
    height                   uint16
    pixels                   width * height * 3 bytes, RGB, row-major
```

Pages are rendered with a DPI (dots per inch) of 150, which strikes the best balance between OCR accuracy, document size, and readability.

The host validates every value before using it:

Value	Accepted range	Error otherwise
`n_pages`	1 to `MAX_PAGES` (10000)	`MaxPagesException`
`width`	1 to `MAX_PAGE_WIDTH` (10000)	`MaxPageWidthException`
`height`	1 to `MAX_PAGE_HEIGHT` (10000)	`MaxPageHeightException`

A short read (fewer bytes than announced) raises `ConverterProcException`. After the last page, the host closes standard output and ignores anything else. Each page's pixels are compressed and turned into a PDF page on the host (with OCR if requested) as soon as they arrive, so memory use grows slowly with the document size.

Standard error

Standard error carries the sandbox's own diagnostics. It can be enriched with debug information from gVisor and Podman by passing the `--debug` flag. Since it's untrusted, it is sanitized as well and any control characters are stripped before writing to the terminal.

Exit codes

The container communicates failures through its exit code. Conversion errors start at `ERROR_SHIFT = 128`:

Exit code	Meaning
128	Unspecified conversion error
138	The document format is not supported
144	HWP / HWPX formats are not supported in Qubes

Exit code	Meaning
148	Conversion to PDF with LibreOffice failed
158	The document appears to be corrupted and could not be opened
168	Page-related error (generic)
169	Number of pages could not be extracted from PDF
170	Number of pages exceeds maximum
172	A page exceeded the maximum width
173	A page exceeded the maximum height
174	Page count mismatch between the document and the rendered pages
228	Some unexpected error occurred while converting the document
126	Qubes OS only: the qrexec call failed

Any other code maps to "Unknown error code". The host-side definitions are in `dangerzone/conversion_errors.py`.

Versioning

The image is published as `ghcr.io/freedomofpress/dangerzone/v1`. The `v1` suffix is the protocol version: a change that breaks this interface will be published under `v2`, and a Dangerzone release will switch to it. See Independent sandbox updates.

Settings

Dangerzone stores its settings in a JSON file named `settings.json`. The graphical application writes it whenever you change a setting, and reads it on start. Missing keys are filled with their defaults, so you can delete the file to reset everything.

Location

Platform	Path
Linux	`~/.config/dangerzone/settings.json`
macOS	`~/Library/Application Support/dangerzone/settings.json`
Windows	`%LOCALAPPDATA%\dangerzone\dangerzone\settings.json`

The directory is the per-user configuration directory returned by platformdirs for the `dangerzone` application.

Keys

Conversion

`save`
: Boolean. Default `true`. Only used by the GUI. Whether to save the safe PDF to disk. When set to `false`, the safe PDF will be output in a temporary directory instead.

`archive`
: Boolean. Default `true`. Only used by the GUI. Move the original document into an `unsafe/` subdirectory after a successful conversion. Corresponds to the "Move original documents to 'unsafe' subdirectory" choice in the GUI.

`safe_extension`
: String. Default `"-safe.pdf"`. Suffix appended to the original name to form the output name. Must end with `.pdf`. Ignored if save is `false`.

`output_dir`
: String or `null`. Default `null`. Directory where safe PDFs are saved when "Save safe PDFs to" is selected. `null` means next to the original.

`ocr`
: Boolean. Default `true`. Only used by the GUI. Run OCR on the safe PDF.

`ocr_language`
: String. Default `"English"`. Display name of the OCR language, as listed in `share/ocr-languages.json`.

`open`
: Boolean. Default `true`. Open the safe PDF after conversion (only used by the GUI).

`open_app`
: String or `null`. Default `null`. On Linux, the desktop application used to open the safe PDF. `null` means the default PDF viewer (only used by the GUI).

Container runtime

`container_runtime`
: String, absent by default. Full path of the container runtime to use instead of the platform default. Set with `dangerzone-cli --set-container-runtime` and removed with `--set-container-runtime default`. See Use Podman Desktop or a custom runtime.

`stop_other_podman_machines`
: String. Default `"ask"`. What to do on macOS and Windows when another Podman machine is already running, since only one can run at a time. `"ask"` prompts, `"always"` stops the other machine, `"never"` makes Dangerzone quit instead. The prompt's "Remember my choice" checkbox sets this to `"always"` or `"never"`.

Updates

`updater_check_all`
: Boolean or `null`. Default `null`. Whether to check for container updates and new releases. `null` means the user has not been asked yet. Replaces the older `updater_check` key.

`updater_ask_before_download`
: Boolean. Default `true`. Ask before downloading a new sandbox image when one is available. The "Always download sandbox updates" checkbox in that prompt sets it to `false`.

`updater_last_check`
: Integer or `null`. Default `null`. Last update check, in seconds since the Unix epoch. `null` means Dangerzone has never checked. Checks are spaced at least 12 hours apart.

`updater_latest_version`
: String. Default: the running version. Latest release version that the updater has seen.

`updater_latest_changelog`
: String. Default `""`. Changelog of the latest release seen, rendered in the "New version available" dialog.

`updater_remote_log_index`
: Integer. Default `0`. Rekor transparency log index of the latest sandbox image seen in the registry. Used to detect that a newer image exists.

`updater_errors`
: Integer. Default `0`. Number of consecutive update check errors. Reset to `0` on the next successful check.

See Update notifications for how these keys drive the update flow.

Related files

Next to the settings, Dangerzone keeps the sandbox image signatures under the per-user data directory (`~/.local/share/dangerzone/signatures` on Linux, `~/Library/Application Support/dangerzone/signatures` on macOS, `%LOCALAPPDATA%\dangerzone\dangerzone\signatures` on Windows), including the `last_log_index` file that `dangerzone-image load-archive` checks before installing an archive. See Directories for all the data that Dangerzone stores.

Environment variables

These variables change how Dangerzone behaves. Most of them exist for development and testing. None of them is needed for normal use.

`DANGERZONE_DEV`
: Set to `1` to run Dangerzone from a source tree. Resources are looked up under `share/` in the checkout instead of the installed locations, and development-only behaviour is enabled (such as the hidden `--unsafe-dummy-conversion` CLI flag and `QUBES_CONVERSION`). All the build from source guides set it.

`DANGERZONE_BYPASS_SIG_CHECKS`
: Only honoured together with `DANGERZONE_DEV=1`. Set to `1` to skip the Cosign signature verification of the sandbox image. Meant for testing a locally built, unsigned image stored in `share/container.tar`. See Use a local container image.

`DANGERZONE_INSECURE_CONVERTER_PATH`
: Qubes OS development only. Path to a checkout of the `dangerzone-image` repository (its `src` directory). The server-side conversion code is sent to the disposable qube through the `dz.ConvertDev` RPC call on each conversion, so changes are picked up without rebuilding the RPM. See Build from source on Qubes OS.

`QUBES_CONVERSION`
: Qubes OS development only, and only honored together with `DANGERZONE_DEV=1`. Set to `1` to convert documents in disposable qubes instead of containers when running from source inside a qube. Installed Qubes builds detect this on their own.

`SIGSTORE_REKOR_PUBLIC_KEY`
: Path to a Rekor public key that replaces the one bundled in `share/rekor.pub`, used to verify the transparency log entries of the sandbox image signatures. Only needed if Sigstore rotates its key before a Dangerzone release ships the new one. See Configure the verification material.

Variables set by Dangerzone

Dangerzone also sets a few variables for the processes it starts. They are listed here so that they are not a surprise:

`OMP_NUM_THREADS`, `OMP_THREAD_LIMIT`
: Limit the threads used by Tesseract during OCR.

`QT_MAC_WANTS_LAYER`
: Set on macOS for the Qt graphical toolkit.

`IS_WORKER_PROCESS`
: Marks the OCR worker processes spawned by the application.

Directories

This page lists the directories Dangerzone uses to store data, on each platform.

Configuration directory

macOS

📁 **`~/Library/Application Support/dangerzone/`**

Same as the data directory.

Windows

📁 **`%LOCALAPPDATA%\dangerzone\dangerzone\`**

Same as the data directory.

Linux

📁 **`~/.config/dangerzone/`**

Respects the `XDG_CONFIG_HOME` environment variable.

`settings.json`
: The Dangerzone settings. See Settings.

Data directory

macOS

📁 **`~/Library/Application Support/dangerzone/`**

Same as the configuration directory.

Windows

📁 **`%LOCALAPPDATA%\dangerzone\dangerzone\`**

Same as the configuration directory.

Linux

📁 **`~/.local/share/dangerzone/`**

> Respects the `XDG_DATA_HOME` environment variable.

`signatures/`
: The signatures of the sandbox images that Dangerzone has verified, grouped by the digest of the public key that signed them. See Independent sandbox updates.

`signatures/last_log_index`
: The Rekor transparency log index of the installed sandbox image. Dangerzone uses it to refuse older images, and considers the sandbox image as not installed if this file is missing.

Cache directory

> **macOS**
>
> 📁 **`~/Library/Caches/dangerzone/`**

> **Windows**
>
> 📁 **`%LOCALAPPDATA%\dangerzone\dangerzone\Cache\`**

> **Linux**
>
> 📁 **`~/.cache/dangerzone/`**
>
> Respects the `XDG_CACHE_HOME` environment variable. Dangerzone does not use this directory on Linux.

Dangerzone regenerates these files as needed, so you can safely delete them.

`containers.conf`
: The Podman configuration that Dangerzone passes to its bundled Podman via the `CONTAINERS_CONF` environment variable. It is rewritten whenever Dangerzone runs.

`shared/seccomp.gvisor.json`
: A copy of the seccomp profile of the sandbox. Dangerzone mounts the cache directory read-only into the Podman machine, so that the profile can reach the container.

Signing keys

Dangerzone uses several keys to sign what it ships. This page lists them so that you can cross-check fingerprints from more than one place.

Release key (PGP)

Our binaries, source archives, container archives, and checksums are signed with a PGP key owned by Freedom of the Press Foundation.

Name	Dangerzone Release Key
Email	`dangerzone-release-key@freedom.press`
Fingerprint	`DE28 AB24 1FA4 8260 FAC9 B8BA A7C9 B385 2260 4281`
Signing subkey	`04CA BEB5 DD76 BACF 2BD4 3D2F F3AC C60F 62EA 51CB`
Download	keys.openpgp.org

The same key signs the APT and RPM repositories at `packages.freedom.press`, which is why the fingerprint above is the one to confirm when `dnf` asks `Importing GPG key 0x22604281`.

Cross-check the fingerprint with the footer of dangerzone.rocks and the bio of our Mastodon account. How to use it: Verify PGP signatures.

Sandbox image key (Cosign)

The sandbox container image is signed with Cosign. The public key is shipped with every Dangerzone installation as `share/freedomofpress-dangerzone.pub`, next to a detached PGP signature of it (`freedomofpress-dangerzone.pub.asc`) made with the release key above. Dangerzone verifies every sandbox image against this key before using it. See Independent sandbox updates.

Signatures are recorded in the Sigstore transparency log (Rekor). Dangerzone bundles and pins the Rekor public key as `share/rekor.pub`. It can be overridden with `SIGSTORE_REKOR_PUBLIC_KEY`, see environment variables.

Platform code signing

- **macOS**: the application bundle and `.dmg` are signed with the `Developer ID Application: Freedom of the Press Foundation (94ZZGGGJ3W)` certificate and notarized by Apple.
- **Windows**: `dangerzone.exe`, `dangerzone-cli.exe`, and the `.msi` installer are signed with a code-signing certificate owned by Freedom of the Press Foundation and provided by Azure Trusted Signing.

These signatures are checked by the operating system. The PGP signatures provide an independent, second check.

Security Policy

The Dangerzone team welcomes feedback from security researchers and the general public to help improve our security. If you believe you have discovered a security issue, we want to hear from you. This policy outlines steps for reporting vulnerabilities to us, what we expect, what you can expect from us.

Reporting a vulnerability

Please do not report security vulnerabilities through public GitHub issues.

Our preferred communication channels are:

- **Signal:** Not yet publicly available. Please contact us via email and we will provide you with Signal contact details for secure communication.
- **Email:** support@dangerzone.rocks.

A member of our team should acknowledge your report **within 2 weeks**. If we fail to do so, please use a different communication channel.

Once we have acknowledged the report, we ask for **at least 6 weeks** to investigate, implement, and release a fix before any public disclosure, unless we mutually agree on a different timeline. During that time we will keep you updated as we make progress, and we will be available for any question you may have. If we determine that a coordinated public disclosure is necessary before a fix is ready, we will work with you on messaging and timing.

What's in scope

Dangerzone has two main security goals:

- Malicious documents should not infect the user's device, or communicate with other machines.
- All metadata should be destroyed after the conversion process.

Any vulnerability that undermines these two goals **is considered critical** and we advise you to report it via Signal.

About CVEs in our container image

Dangerzone uses several third-party tools to sanitize documents, such as LibreOffice and PyMuPDF. Because these tools have a large attack surface, Dangerzone operates under the assumption that a 0-day vulnerability probably exists for them. For this reason, Dangerzone's primary defense is to isolate these tools within unprivileged, networkless containers using gVisor. Read more in our blog.

Our second line of defense is to make sure our container image is not affected by known vulnerabilities, i.e., CVEs. We have nightly security scans for Critical CVEs, and biweekly security scans for High CVEs. You can see the results of our security scans in https://

cves.dangerzone.rocks. We aim for a 4 week update cadence of our container image, or earlier, if a security finding necessitates it.

If you have encountered a CVE in our container image that violates the above policy, please report it to us.

Security Advisories

When necessary, we have issued CVEs for Dangerzone and security advisories to our users. We are committed to transparency and will continue to issue CVEs and security advisories whenever a finding warrants it.

Security advisories

When necessary, we issue security advisories to our users, in addition to CVEs on GitHub. Advisories are published here and linked from the release notes of the release that addresses them.

Vulnerabilities in the tools inside the sandbox are expected. On their own they do not compromise Dangerzone, since the sandbox contains them. We still publish advisories and updated sandbox images for them, so that a second vulnerability (a sandbox escape) can't be combined with a known one. See Security model.

Date	Summary	Fixed in
2024-12-24	gst-plugins-base vulnerabilities (CVE-2024-47538, CVE-2024-47607, CVE-2024-47615) inside the sandbox	0.8.1
2023-12-07	GhostScript vulnerability (CVE-2023-43115) inside the sandbox	0.5.1
2023-10-25	Missing `default_dispvm` setting in the Qubes OS installation instructions	0.5.1

To report a vulnerability, follow the security policy.

Security Advisory 2024-12-24

In Dangerzone, a security vulnerability was detected in the quarantined environment where documents are opened. Vulnerabilities like this are expected and do not compromise the security of Dangerzone. However, in combination with another more serious vulnerability (also called container escape), a malicious document may be able to breach the security of Dangerzone. We are not aware of any container escapes that affect Dangerzone. **To reduce that risk, you are strongly advised to update Dangerzone to the latest version**.

Summary

A series of vulnerabilities in gst-plugins-base (CVE-2024-47538, CVE-2024-47607 and CVE-2024-47615) affects the **contained** environment where the document rendering takes place.

If one attempts to convert a malicious file with embedded Vorbis or Opus media elements, arbitrary code may run within that environment. Such files look like regular Office documents, which means that you cannot avoid a specific extension. Other programs that open Office documents, such as LibreOffice, are also affected, unless the system has been upgraded in the meantime.

How does this impact me?

The expectation is that malicious code will run in a container without Internet access, meaning that it won't be able to infect the rest of the system.

If you are running Dangerzone via the Qubes OS, you are not impacted.

What do I need to do?

You are **strongly** advised to update your Dangerzone installation to 0.8.1 as soon as possible.

Security Advisory 2023-12-07

In Dangerzone, a security vulnerability was detected in the quarantined environment where documents are opened. Vulnerabilities like this are expected and do not compromise the security of Dangerzone. However, in combination with another more serious vulnerability (also called container escape), a malicious document may be able to breach the security of Dangerzone. We are not aware of any container escapes that affect Dangerzone. **To reduce that risk, you are strongly advised to update Dangerzone to the latest version**.

Summary

A security vulnerability in GhostScript (CVE-2023-43115) affects the **contained** environment where the document rendering takes place. If one attempts to convert a malicious file with an embedded PostScript image, arbitrary code may run within that environment. Such files look like regular Office documents, which means that you cannot avoid a specific extension. Other programs that open Office documents, such as LibreOffice, are also affected, unless the system has been upgraded in the meantime.

How does this impact me?

The expectation is that malicious code will run in a container without Internet access, meaning that it won't be able to infect the rest of the system.

What do I need to do?

You are **strongly** advised to update your Dangerzone installation to 0.5.1 as soon as possible.

Please note that we have recently enabled security scans for our software, and we aim to alert people even sooner about vulnerabilities like these.

Security Advisory 2023-10-25

For users testing our new Qubes integration (beta), please note that our instructions were missing a configuration detail for disposable VMs which is necessary to fully harden the configuration.

These instructions apply to users who followed the setup instructions **before October 25, 2023**.

What you need to do: run the following command in dom0:

```
qvm-prefs dz-dvm default_dispvm ''
```

Explanation: In Qubes OS, the default template for disposable VMs is network-connected. For this reason, we instruct users to create their own disposable VM (`dz-dvm`). However, adversaries with the ability to execute commands on `dz-dvm` would also be able to open new disposable VMs with the default settings. By setting the default_dispvm to "none" we prevent this bypass.

Changelog

All notable changes to this project will be documented in this file.

The format is based on Keep a Changelog since 0.4.1, and this project adheres to Semantic Versioning.

Unreleased

0.11.1

Changes

- We no longer render the Github release notes from Markdown to HTML within the Dangerzone application. This unfortunately makes release notes less readable, but most importantly, it reduces our application's attack surface. (1542)
- Disable cgroups to support systems with restrictive umasks (1574)

Added

- Accept documents from stdin, and write safe PDFs to stdout, by passing `-` as the input filename and `--output-filename -` for the output, enabling standard Unix pipe workflows like `cat file.pdf | dangerzone-cli - -o - > safe.pdf` (#1522)
- (Linux) Show a progress bar when downloading the sandbox (#1164)

Fixed

- Accept OCI image indexes as multi-arch container images, in addition to Docker manifest lists. BuildKit 0.31.0 and later pushes OCI image indexes by default, which made `dangerzone-image prepare-archive` fail with `InvalidMutliArchImage` against newly published images (#1534).
- Respect's user choice to disable or skip update checks (#1517, #1543)
- Alert users when a file can't be processed due to a permission error (#1513)
- Build Qubes packages from tagged released of the https://github.com/freedomofpress/dangerzone-image repo (#1545)
- Fix two small copy issues in the wording of the "download sandbox" dialog (#1578)

Platform changes

- Add support for Fedora 45 (#1568)
- Add support for Ubuntu 26.10 (Stonking Stingray) (#1569)

Development changes

- The documentation is now published at https://docs.dangerzone.rocks (#903)
- Add a security policy for our project (#1278)
- Set the severity cutoff of our code scans from Critical to High, on a biweekly basis (#1438)

0.11.0

Changes

- Linux packages dropped the `container.tar` file. It is expected to be downloaded at the first usage, and then kept up to date. If you prefer to use packages with the `container.tar`, use the `dangerzone-full` variant (#1069)
- **Breaking change (Linux CLI)**: with the slim `dangerzone` package, the CLI no longer auto-downloads the conversion sandbox. New installations must initialize the sandbox once with `dangerzone-image upgrade` (or run the GUI, which prompts for the download) before `dangerzone-cli` can convert documents. Users of the `dangerzone-full` package or who already have an image installed are unaffected.

Fixed

- Remember user-selected output directory across sessions (#1123)
- Fix default PDF reader selection on Linux (#814)
- Update the theme correctly on dark/light mode changes (#1238)
- Update Linux `.desktop` file with missing MIME types for HWP and ODF formats (#646)
- Elide long filenames in the filename label to prevent UI elements from being pushed off-screen (#143)
- Fix macOS app bundle build, which crashed at startup with `module 'dangerzone' has no attribute 'main'` after the CLI parsing refactor (#1470)

Added

- Dangerzone is able to function without a bundled `container.tar` file (#1400)
- Look for desktop entries in `XDG_DATA_DIRS` paths on Linux (#1413)
- It is now possible to convert another set of documents after a first batch (#549), or add more documents to an ongoing conversion (#1464)
- OCR tasks are now queued, resulting in up to x6 speedup (#1329)

Platform changes

- Add support for Ubuntu 26.04 (Resolute Raccoon) (#1467)
- Drop support for Ubuntu 25.04 as it has reached end of life (#1468)
- Add a deprecation warning for Windows 10 (#1357)
- Add support for Fedora 44 (#1462)
- Drop support for Fedora 41 and Fedora 42 as they have reached end of life
- Drop support for Debian bullseye (#1482)

Development changes

- Run macOS Intel CI tests only on scheduled/manual runs to reduce PR CI time (#1338)
- Pin Podman to v5.x.x, to support Windows 10 installations for one last release (#1357)
- Move the conversion error definitions out of the now-removed `conversion/` package and into `dangerzone.conversion_errors`, so the host code keeps a single source of truth for the error codes raised inside the sandbox.
- Move the code for the container image into a separate `freedomofpress/dangerzone-image` repository. CI now consumes the published image via `dangerzone-image prepare-archive`

instead of building it in-tree, and the resulting `container.tar` is cached against the registry digest.
- Drop the format-specific conversion tests and their sample assets. The remaining tests cover the host-side isolation provider interface only; document-format coverage now lives alongside the container image in `freedomofpress/dangerzone-image`.
- Narrow the isolation provider tests to the host/sandbox interface (max-pages and max-dimensions client enforcement, error propagation), using a single canonical sample PDF.
- Run the GUI tests headless by default via `QT_QPA_PLATFORM=offscreen` and drop the `xvfb`/`xvfb-run` setup from CI. A new `--onscreen` pytest flag re-enables the on-screen mode for local debugging.
- Run pytest with `pytest-xdist` and group container-backed and GUI tests into dedicated `xdist_group`s so each group runs on a single worker, avoiding contention over the container runtime and the offscreen Qt platform.
- Pin `XDG_DATA_DIRS` in the GUI logic tests that walk it for `.desktop` files, so the tests don't pick up arbitrary entries from the developer's or CI runner's environment.
- Bump `vfkit` to 0.6.3 (now signed upstream, switching to `vfkit-unsigned` with a relaxed `>=0.6.1` pin) and `cosign` to 2.6.3 in `mazette.lock`, and regenerate `poetry.lock` with Poetry 2.3.
- Update the build instructions for Qubes, in order to work with the new `dangerzone-insecure-converter-qubes` RPM package. (#1463)
- Drop the `cx_Freeze < 8.5.0` workaround now that the upstream MSI build bug has been fixed (#1443)
- Extend `dev_scripts/env.py` to create end-user Dangerzone environments from our APT/YUM repos, with the `--qa` / `--prod` flags

0.10.0 - 2025-12-02

Added

- Sign the sandbox/container images and automatically upgrade them to their latest version (#1006). Read more about this feature in our docs.
- Make Dangerzone use an embedded version of Podman under the hood (#1145)
- Bundle Podman images for Windows and macOS alongside our application (#1170)
- Introduce a new CLI helper called `dangerzone-machine` to manage the Podman machine the Dangerzone uses under the hood (#1172)
- Capture all the command outputs in the logs (#1236)
- Make the `podman pull` command and registry lookups go through Tor on Tails (#1335)
- Add a startup task that installs WSL (#1312)

Removed

- Docker Desktop is no longer required to run Dangerzone. In fact, they are no longer compatible, due to some changes in the bundled container image. Instead, Podman Desktop is used under the hood (#118)

Fixed

- Fix a Dangerzone error that manifested in recent Debian-based environments that included both PySide6 and PySide2 libraries (#1218)

- Issue templates have been updated to work on Windows (#1237)
- Question dialogs now show a question mark instead of an alert (#1198)

Changed

- Update our container image from Debian Bookworm to Debian Trixie (#1243)
- Upgrade the bundled LibreOffice version from 7.x to 25.x, as a result of our Debian Trixie switch (#1165)

Platform changes

- Add support for Ubuntu 25.10 (Questing Quokka) (#1264)
- Add support for Fedora 43 (#1263)
- Add support for the new Debian Testing (Forky) (#1265)
- Drop support for Ubuntu Oracular (24.10) since it is end of life (#1246)
- Add a deprecation warning for Debian bullseye (#1214)

Development changes

- Use the archived backports for Debian bullseye (#1213)
- Install `numpy` 2.0.0 in Python 3.9 envs, and use more recent `numpy` in Python environments >= 3.10, to avoid some compatibility issues with Python 3.13 (#1206)
- Improve our release instructions by splitting the large `RELEASE.md` file into distinct docs, whose instructions can be executed sequentially (#1212)
- Run our full CI test suite on Windows and macOS GitHub runners (#1009)
- Add end to end functional tests for independent container updates (#1157)
- Add developer instructions for releasing container images independently (#1162)
- Do not check the Git commit/branch when attesting provenance info (#1270)

0.9.1 - 2025-07-10

Fixed

- Enforce passing our own seccomp profile when running the sandbox, to avoid a regression that has manifested since Docker Desktop 4.42.0 (#1191)
- Fix a conversion failure when a user has enabled Podman Desktop, whereby the Podman VM cannot find the necessary seccomp profile (#1187)
- Make our seccomp policy allow unknown syscalls for podman versions < 4.0 (#1201)

Changed

- Upgrade the Python version we ship in Windows / macOS to 3.13 (#1120)
- (Docs) Update installation instructions for Fedora. `dnf config-manager` is not a plugin (#1176)
- (Docs) Update installation instructions (and CI checks) for Debian derivatives (#1141, #1163)
- (Docs) Point to relevant install sections in the OS-support table (#1160)
- (Docs) Remove the need to create a `containers.conf` file, when integrating Dangerzone with Podman Desktop (#1127)

Removed

- Platform support: Drop support for Fedora 41 as security support has ended (#1178)

Development changes

- Vendor GitHub assets using the `mazette` tool (see #1146 for the original implementation, renamed later from `assets` to `mazette`)
- Use a newer `cx_Freeze` version that fixes an issue with bundling PyMuPDF (1128)

0.9.0 - 2025-04-09

Added

- Platform support: Add support for Fedora 42 (#1091)
- Platform support: Add support for Ubuntu 25.04 (Plucky Puffin) (#1090)
- (experimental): It is now possible to specify a custom container runtime in the settings, by using the `container_runtime` key. It should contain the path to the container runtime you want to use. Please note that this doesn't mean we support more container runtimes than Podman and Docker for the time being, but enables you to chose which one you want to use, independently of your platform. (#925)
- Document Operating System support #986
- Tests: Look for regressions when converting PDFs #321
- Ensure container image reproducibilty across different container runtimes and versions (#1074)
- Implement container image attestations (#1035)
- Inform user of outdated Docker Desktop Version (#693)
- Add support for Python 3.13 (#992)
- Publish the built artifacts in our CI pipelines (#972)

Fixed

- Fix our Debian Trixie installation instructions using Sequoia PGP (#1052)
- Fix the way multiprocessing works on macOS (#873)
- Update minimum Docker Desktop version to fix an stdout truncation issue (#1101)

Removed

- Platform support: Drop support for Ubuntu Focal, since it's nearing end-of-life (#1018)
- Platform support: Drop support for Fedora 39 (#999)

Changed

- Switch base image to Debian Stable (#1046)
- Track image tags instead of image IDs in `image-id.txt` (#1020)
- Migrate to Wix 4 (windows building tool) (#602). Thanks @jkarasti for the contribution.
- Add a `--debug` flag to the CLI to help retrieve more logs (#941)
- The `debian` base image is now fetched by digest. As a result, your local container storage will no longer show a tag for this dependency (#1116). Thanks @sudoforge for the contribution.

- The `debian` base image is now referenced with a fully qualified URI, including the registry hostname (#1118). Thanks @sudoforge for the contribution.
- Update the Dangerzone container image and its dependencies (gVisor, Debian base image, H2Orestart) to the latest versions:
- Debian image release: `bookworm-20250317-slim@sha256:1209d8fd77def86ceb6663deef7956481cc6c14a25e1e64daec12c0ceffcc19d`
- Debian snapshots date: `2025-03-31`
- gVisor release date: `2025-03-26`
- H2Orestart plugin: `v0.7.2` (`d09bc5c93fe2483a7e4a57985d2a8d0e4efae2efb04375fe4b59a68afd7241e2`)

Development changes

- Make container image scanning work for Silicon macOS (#1008)
- Automate the main bulk of our release tasks (#1016)
- CI: Enforce updating the CHANGELOG in the CI (#1108)
- Add reference to funding.json (required by floss.fund application) (#1092)
- Lint: add ruff for linting and formatting (#1029). Thanks @jkarasti for the contribution.
- Work around a `cx_freeze` build issue (#974)
- tests: mark the hancom office suite tests for rerun on failures (#991)
- Update reference template for Qubes to Fedora 41 (#1078)

0.8.1 - 2024-12-24

- Update the container image

Added

- Disable gVisor's DirectFS feature (#226). Thanks EtiennePerot for the contribution.

Removed

- Platform support: Drop support for Fedora 39, since it's end-of-life (#999)

Updated

- Bump `slsa-framework/slsa-github-generator` from 2.0.0 to 2.1.0 (#1109)

Development changes

Thanks @jkarasti for the contribution.

- Automate a large portion of our release tasks with `doit` (#1016)

0.8.0 - 2024-11-06

Added

- Point to the installation instructions that the Tails team maintains for Dangerzone (announcement)
- Installation and execution errors are now caught and displayed in the interface (#193)

- Prevent users from using illegal characters in output filename (#362). Thanks @bnewc for the contribution!
- Add support for Fedora 41 (#947)
- Add support for Ubuntu Oracular (24.10) (#954)

Fixed

- Update our macOS entitlements, removing now unneeded privileges (#638)
- Make Dangerzone work on Linux systems with SELinux in enforcing mode (#880)
- Process documents with embedded multimedia files without crashing (#877)
- Search for applications that can read PDF files in a more reliable way on Linux (#899)
- Handle and report some stray conversion errors (#776). Thanks @amnak613 for the contribution!
- Replace occurrences of the word "Docker" in Podman-related error messages in Linux (#212)

Changed

- The second phase of the conversion (pixels to PDF) now happens on the host. Instead of first grabbing all of the pixel data from the first container, storing them on disk, and then reconstructing the PDF on a second container, Dangerzone now immediately reconstructs the PDF **on the host**, while the doc to pixels conversion is still running on the first container. The sanitation is no less safe, since the boundaries between the sandbox and the host are still respected (#625)
- PyMuPDF is now vendorized for Debian packages. This is done because the PyMuPDF package from the Debian repos lacks OCR support (#940)
- Always use our own seccomp policy as a default (#908)
- Debian packages are now amd64 only, which removes some warnings in Linux distros with 32-bit repos enabled (#394)
- Allow choosing installation directory on Windows platforms (#148). Thanks @jkarasti for the contribution!
- Bumped H2ORestart LibreOffice extension to version 0.6.6 (#943)
- Platform support: Ubuntu Focal (20.04) is now deprecated, and support will be dropped with the next release (#965)

Removed

- Platform support: Drop Ubuntu Mantic (23.10), since it's end-of-life (#977)

Development changes

- Build Debian packages with pybuild (#773)
- Test Dangerzone on Intel macOS machines as well (#932)
- Switch from CircleCI runners to Github actions (#674)
- Sign Windows executables and installer with SHA256 rather than SHA1 (#931). Thanks @jkarasti for the contribution!

0.7.1 - 2024-10-01

Fixed

- Fix an `image-id.txt` mismatch happening on Docker Desktop >= 4.30.0 (#933)

0.7.0 - 2024-07-10

Added

- Integrate Dangerzone with gVisor, a memory-safe application kernel, thanks to @EtiennePerot (#126). As a result of this integration, we have also improved Dangerzone's security in the following ways:
- Prevent attacker from becoming root within the container (#224)
- Use a restricted seccomp profile (#225)
- Make use of user namespaces (#228)
- Files can now be drag-n-dropped to Dangerzone (issue #409)

Fixed

- Fix a deprecation warning in PySide6, thanks to @naglis (issue #595)
- Make update notifications work in systems with PySide2, thanks to @naglis (issue #788)
- Updated the Dangerzone container image to use Alpine Linux 3.20 (#812)
- Fix wrong file permissions in Fedora packages (issue #727)
- Quote commands in installation instructions, making it compatible with `zsh` based shells. (issue #805)
- Order the list of PDF viewers and return the default application first on Linux, thanks to @rocodes (issue #814)

Removed

- Platform support: Drop Fedora 38, since it's end-of-life (issue #840)

Development changes

- Bumped the minimum python version to 3.9, due to Pyside6 dropping support for python 3.8 (#780)
- Minor amendments to the codebase (in #811)
- Use the original line ending (usually `LF`) for all content except images (#838)
- Explained how to create, sign, and verify source tarballs (#823)
- Added a design doc for the update notifications
- Added a design doc for the gVisor integration (#815)
- Removed the python shebang from some files

0.6.1 - 2024-05-13

Added

- Platform support: Ubuntu 24.04 and Fedora 40 (issue #762)

Fixed

- Handle timeout errors (`"Timeout after 3 seconds"`) more gracefully (issue #749)
- Make Dangerzone work in macOS versions prior to Ventura (13), thanks to @maltfield (issue #471)
- Make OCR work again in Qubes Fedora 38 templates (issue #737)
- Make .svg / .bmp files selectable when browsing files via the Dangerzone GUI (#722)
- Linux: Show the proper application name and icon for Dangerzone, in the user's window manager, thanks to @naglis (issue #402)
- Linux: Allow opening multiple files at once, when selecting them from the user's file manager, thanks to @naglis (issue #797)
- Linux: Do not include Dangerzone in the list of available PDF viewers, thanks to @naglis (issue #790)
- Linux: Handle filenames with invalid Unicode characters in the Dangerzone CLI, thanks to @naglis (issue #768)

Changed

- Sign our release assets with the Dangerzone signing key, and provide instructions to end-users (issue #761
- Use the newest reimplementation of the PyMuPDF rendering engine (`fitz`) (issue #700)
- Development: Build Dangerzone using the latest Wix 3.14 release (#746

0.6.0 - 2024-02-26

Added

- Platform support: Fedora 39 (issue #606)
- Add new file formats: epub svg and several image formats (BMP, PNM, BPM, PPM) (issue #697)

Fixed

- Fix mismatched between between original document and converted one (issue #626). This does not affect the quality of the final document.
- Capitalize "dangerzone" on the application as well as on the Linux desktop shortcut, thanks to @sudwhiwdh #676
- Fedora (Linux): Add missing Dangerzone logo on application launcher (issue #645)
- Prevent document conversion from failing due to lack of space in the converter. This affected mainly systems with low computing resources such as Qubes OS (issue #574)
- Add a missing dependency to our Apple Silicon container image, which affected dev environments only, thanks to @prateekj117 (#671)
- Development: Add missing check when building container image, thanks to @EtiennePerot (#721)

Changed

- Feature: Add support for HWP/HWPX files (Hancom Office) for macOS Apple Silicon devices (issue #498, thanks to @OctopusET)

- Replace Dangerzone document rendering engine from pdftoppm PyMuPDF, essentially replacing a variety of tools (gm / tesseract / pdfunite / ps2pdf) (issue #658)
- Changed project license from MIT to AGPLv3 (related to issue #658)
- Containers: stream pages instead of mounting directories. For users in practice this doesn't change much, but it opens up technical possibilities that go from security to usability. (issue #443)
- Ubuntu Jammy (Linux): add external depedency (provided by the Dangerzone repository) which fixes podman crashing during standar stream I/O (issue #685)

Removed

- Removed timeouts (issue #687)
- Platform support: Drop Ubuntu 23.04 (Lunar Lobster), since it's end-of-life (issue #705)

0.5.1 - 2023-12-08

Fixed

- Our Qubes RPM package was missing critical dependencies for the conversion of a document from pixels to PDF (issue #647)

Changed

- Use more descriptive button labels in update check prompt (issue #527, thanks to @garrettr)

Removed

- Platform support: Drop Fedora 37, since it reached end-of-life (issue #637)

Security

- Security advisory 2023-12-07: Protect our container image against CVE-2023-43115, by updating GhostScript to version 10.02.0.
- Security advisory 2023-10-25: prevent dz-dvm network via dispVMs. This was officially communicated on the advisory date and is only included here since this is the first release since it was announced.

0.5.0 - 2023-10-19

Added

- Platform support: Beta integration with Qubes OS (issue #412)
- Platform support: Ubuntu 23.10 (Mantic Minotaur) (issue #601)
- Add client-side timeouts in Qubes (issue #446)
- Add installation instructions for Qubes (issue #431)
- Development: Add tests that run Dangerzone against a pool of roughly 11K documents (PR #386)
- Development: Grab the output of commands when in development mode (issue #319)

Fixed

- Fix a bug that was introduced in version 0.4.1 and could potentially lead to excluding the last page of the sanitized document (issue #560)
- Fix the parsing of a document's page count (issue #565)
- Platform support: Fix broken Dangerzone upgrades in Fedora (issue #514)
- Make progress reports in Qubes real-time (issue #557)
- Improve the handling of various runtime errors in Qubes (issue #430)
- Pass OCR parameters properly in Qubes (issue #455)
- Fix dark mode support (issue #550, thanks to @garrettr)
- MacOS/Windows: Sync "Check for updates" checkbox with the user's choice (issue #513)
- Fix issue where changing document selection to a file from a different directory would lead to an error (issue #581)
- Qubes: in the cli version "Safe PDF created" would be shown twice (issue #555)
- Qubes: in the cli version the percentage is now rounded to the unit (issue #553)
- Qubes: clean up temporary files (issue #575)
- Qubes: do not open document if the conversion failed (issue #581)
- Development: Switch from the deprecated `bdist_rpm` toolchain to the more modern RPM SPEC files, when building Fedora packages (issue #298)
- Development: Make our dev scripts properly invoke Docker in MacOS / windows (issue #519)

Changed

- Shave off ~300MiB from our container image, using the fast variant of the Tesseract OCR language models (issue #545)
- When a user is asked to enable updates, make "Yes" the default option (issue #507)
- Use Fedora 38 as a template in our Qubes build instructions (PR #533)
- Improve the installation docs for newcomers (issue #475)
- Development: Explain how to get the application password from the MacOS keychain (issue #522)

Removed

- Remove the `dangerzone-container` executable, since it was not used in practice by any user (PR #538)

Security

- Do not allow attackers to show error or log messages to Qubes users (issue #456)

0.4.2 - 2023-08-08

Added

- Inform about new updates on MacOS/Windows platforms, by periodically checking our GitHub releases page (issue #189)
- Feature: Add support for HWP/HWPX files (Hancom Office) (issue #243, thanks to @OctopusET)

- **NOTE:** This feature is not yet supported on MacOS with Apple Silicon CPU or Qubes OS (issue #494, issue #498)
- Allow users to change their document selection from the UI (issue #428)
- Add a note in our README for MacOS 11+ users blocked by SIP (PR #401, thanks to @keywordnew)
- Platform support: Alpha integration with Qubes OS (issue #411)
- Platform support: Debian Trixie (13) (issue #452)
- Platform support: Ubuntu 23.04 (Lunar Lobster) (issue #453)
- Development: Use Qt6 in our CI runners and dev environments (issue #482)

Removed

- Platform support: Drop Fedora 36, since it's end-of-life (issue #420)
- Platform support: Drop Ubuntu 22.10 (Kinetic Kudu), since it's end-of-life (issue #485)

Fixed

- Add missing language detection (OCR) models (issue #357)
- Replace deprecated `pipes` module with `shlex` (issue #373, thanks to @OctopusET)
- Shrink container image with `--no-cache` option on `apk` (issue #459, thanks to @OctopusET)

Security

- Continuously scan our Python dependencies and container image for vulnerabilities (issue #222)
- Sanitize potentially unsafe characters from strings that are shown in the GUI/terminal (PR #491)

0.4.1 - 2023-04-24

Added

- Feature: Add version info in the CLI and GUI (issue #219)
- Development: Improve CI stability and coverage (issue #292, issue #217, issue #229)
- Development: Provide dev scripts for testing Dangerzone in a container and running our QA pipeline (issue #286, issue #287)
- Development: Support Dangerzone development on Fedora 37 (issue #294)
- Development: Allow running Mypy on MacOS M1 machines (issue #177)
- Development: Add dummy isolation provider for testing non-conversion-related issues in virtualized Windows and MacOS, where Docker can't run, due to the lack of nested virtualization (issue #229)
- Add support for more MIME types that were previously disregarded (issue #369)
- Platform support: Add support for Fedora 38

Changed

- Full release under Freedom of the Press Foundation: signing keys have changed from the original developer Micah Lee / First Look Media to FPF's signing keys. Linux packages moved from Packagecloud to FPF's servers

- Installation instructions updated to reflect change in key owership to FPF
- Platform support: MacOS (Apple Silicon) native application with significant performance boost (issue #50)
- Feature: Introduce PySide6 / Qt6 support on Windows, MacOS, and Linux (dev-only) (issue #219)
- Feature: Adjust conversion timeouts based on the document's pages/size, and allow users to disable them with `--disable-timeouts` (available when you run the Dangerzone from the terminal) (issue #327)
- Development: Update Linux instructions for development on Qubes

Removed

- Platform support: Drop Fedora 35, since it's end-of-life (issue #308)
- Bug fix: Remove unused PDFtk and sudo libraries from the container image, to lower its attack surface and reduce its size (issue #232)

Fixed

- Feature: Convert documents with non-standard permissions or SELinux labels (issue #335)
- Bug fix: Report exceptions during conversions (issue #309)
- Bug fix: (Windows) Fix Dangerzone description on "Open With" (issue #283)
- Bug fix: Remove document conversion artifacts when conversion fails and store them on volatile memory instead of on a disk directory (issue #317)

Security

- Bug fix: Do not print debug logs in end-user executables (issue #316)

0.4.0 - 2022-12-01

- Platform support: Re-add Fedora 37 support
- Platform support: Add Debian Bookworm (12) support (issue #172)
- Platform support: Reinstate Ubuntu Focal support (issue #206)
- Platform support: Add Ubuntu 22.10 "Kinetic Kudu" support (issue #265)
- Feature: Support bulk conversion to safe PDFs (issue #77)
- Feature: Option to archive unsafe directories (issue #255)
- Feature: Support python 3.10
- Feature: When quitting while still converting, confirm if user is sure
- Bug fix: Fix unit tests on Windows
- Bug fix: Do not hardcode "docker" in help messages, now that Podman is also used (issue #122)
- Bug fix: Failed execution no longer produces an empty "safe" documents (issue #214)
- Bug fix: Malfunctioning "New window" logic was replaced with multi-doc support (issue #204)
- Bug fix: re-adds support for 'open with Dangerzone' from finder on macOS (issue #268)
- Bug fix: (macOS) quit Dangerzone when main window is closed (issue #271)

0.3.2 - 2022-09-06

- Bug fix: some non-ascii characters like “ would prevent Dangerzone from working (issue #144)
- Bug fix: error where Dangerzone would show "permission denied: '/tmp/input_file'" (issue #157)
- Bug fix: remove containers after use, enabling Dangerzone to run after 1000+ converted docs (issue #197)
- Security: limit container capabilities, run in container as non-root and limit privilege escalation (issue #169)

0.3.1 - 2021-12-14

- Bug fix: Allow converting documents on different mounted filesystems than the container volume
- Bug fix: In GUI mode, don't always OCR document
- Bug fix: In macOS, fix "open with" Dangerzone so documents are automatically selected
- Windows: Change packaging to avoid anti-virus false positives

0.3 - 2021-12-01

- Removes the need for internet access by shipping the Dangerzone container image directly with the software
- Friendly user experience with a progress bar
- Support for Macs with M1 chips

0.2.1 - 2021-06-22

- Switch from Docker to Podman for Linux
- Improve CLI colors

0.2 - 2021-06-22

- Command line support and improved terminal output
- Additional container hardening
- Fix macOS crash on quit
- Fix --custom-container CLI argument

0.1.5 - 2021-02-11

- Add support for macOS Big Sur
- Drop support for Ubuntu 19.10

0.1.4 - 2020-10-30

- Suppress confusing stderr output, and fix bug when converting specific documents
- Switch from PyQt5 to PySide2
- Improve Windows and Mac packaging

- Add support for Fedora 32

0.1.3 - 2020-09-25

- Add support for Ubuntu 20.04 LTS (#79)
- Prevent crash in macOS if specific PDF viewers are installed (#75)

0.1.2 - 2020-04-09

- (Linux only)
- Add support for Ubuntu 18.04 LTS

0.1.1 - 2020-03-17

- Fix macOS bug that caused a crash on versions earlier than Catalina
- Fix macOS app bundle ODF extensions (`.ods .odt`)
- Allow Linux users to type their password instead of adding their user to the `docker` group
- Use Docker instead of Podman in Fedora
- Allow the use of either OS-packaged Docker or Docker CE in Linux
- Allow opening `.docm` files
- Allow using a custom container for testing

0.1 - 2020-02-29

- First release

Third-Party Notices

This project may include or interact with third-party software components and services. The following is a list of attributions for these components and related technologies:

Please refer to the respective sources for licensing information and further details regarding the use of these components.

Cosign

Code signing and transparency for containers and binaries.

- License: Apache-2.0
- Repository: https://github.com/sigstore/cosign
- Download link: See `cosign` section in `mazette.lock`
- Included in: Dangerzone installers (Windows MSI, macOS DMG, Linux DEB/RPM)

(Last updated: See date of `mazette.lock`)

gvproxy

A new network stack based on gVisor.

- License: Apache-2.0
- Repository: https://github.com/containers/gvisor-tap-vsock
- Download link: See `gvproxy` section in `mazette.lock`
- Included in: Dangerzone installers (macOS DMG)

(Last updated: See date of `mazette.lock`)

Podman

A tool for managing OCI containers and pods.

- License: Apache-2.0
- Repository: https://github.com/containers/podman
- Download link: See `podman` section in `mazette.lock`
- Included in: Dangerzone installers (Windows MSI and macOS DMG)

(Last updated: See date of `mazette.lock`)

Podman Machine OS disk image

Disk images for Podman Machine VMs.

- License: Apache-2.0 (see https://github.com/containers/podman-machine-os/issues/80)
- Repository: https://github.com/containers/podman-machine-os
- Download link: See `machine-os` section in `mazette.lock`
- Included in: Dangerzone installers (Windows MSI and macOS DMG)

(Last updated: See date of `mazette.lock`)

Tesseract data

Fast integer versions of trained LSTM models.

- License: Apache-2.0
- Repository: https://github.com/tesseract-ocr/tessdata_fast/
- Download link: See `tessdata` section in `mazette.lock`
- Included in: Dangerzone installers (Windows MSI and macOS DMG)

(Last updated: See date of `mazette.lock`)

vfkit

Command-line tool to start VMs on macOS.

- License: Apache-2.0
- Repository: https://github.com/crc-org/vfkit
- Download link: See `vfkit` section in `mazette.lock`
- Included in: Dangerzone installers (macOS DMG)

(Last updated: See date of `mazette.lock`)

Roadmap

Dangerzone's plans are tracked as GitHub milestones. There is no separate roadmap document: the milestones are the source of truth, and they change as work progresses.

Milestone	What it holds
X.Y.Z	0.11.0 for instance here.
1.0.0	What we want in place before calling Dangerzone 1.0.
Backlog	Technical debt that we need to pay off, or important issues that we need to tackle at some point.
Evergreen	The milestone that's always active, tracking long-term changes, not directly related to a specific version.

The milestones page always has the current list, with progress. Larger efforts are tracked as issues labelled epic.

If you want to help, pick up an issue labelled good first issue or help wanted.

Explanation

How Dangerzone works

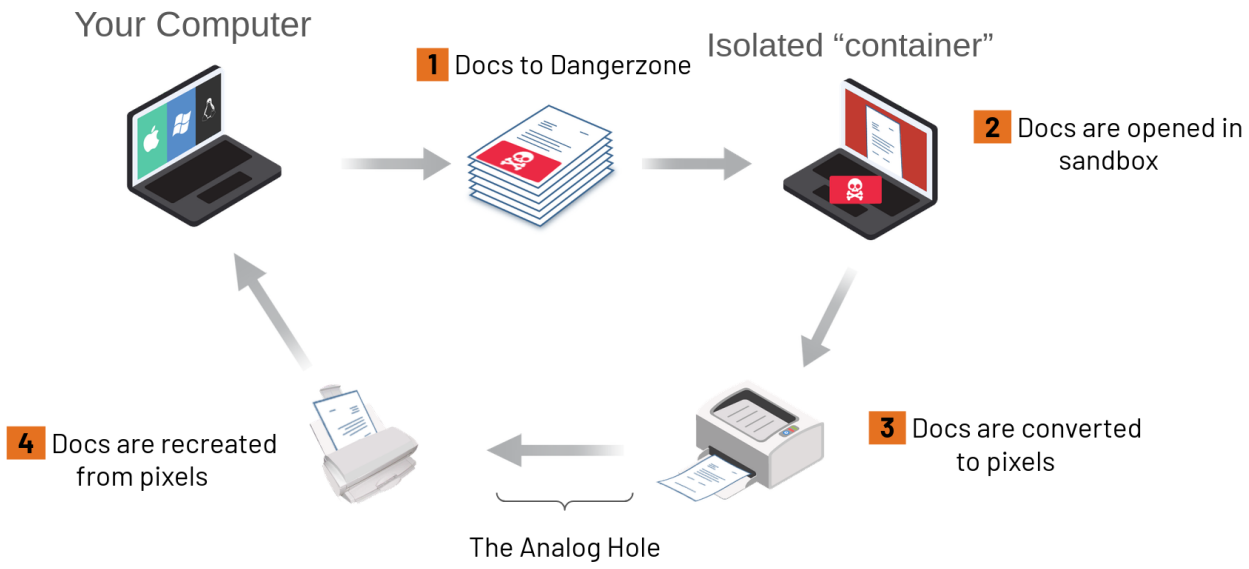
Dangerzone takes potentially dangerous PDFs, office documents, or images and converts them to a safe PDF. But how does that work?

The core idea: flatten everything to pixels

A document format like PDF or DOCX is a small program as much as it is a picture of a page, and controlling what this program can do is hard: some embedded fonts can exploit security holes in the font renderer, for instance, and then escalate to executing code on your machine.

Dangerzone sidesteps this problem entirely. In a secure sandbox, the document will be converted to raw pixel data: a huge list of RGB color values for each page. Then, outside of the sandbox, Dangerzone takes this pixel data and converts it back into a PDF.

Converting to pixel data renders any embedded script ineffective: whatever the original document was hiding doesn't survive this step. Then, the safe PDF is rebuilt from those bitmaps by trusted code on your side of the fence, and so we can be sure it doesn't contain any scripts at all.



Dangerzone was inspired by Qubes trusted PDF, which applies the same idea with virtual machines. Dangerzone brings it to non-Qubes operating systems by using containers as sandboxes.

The sandbox

The sandbox is a container image, built and published separately in the dangerzone-image repository. Dangerzone runs it with Podman:

- On Linux, with the system's Podman.

- On macOS and Windows, with a Podman that is embedded in the application and runs inside a small Linux virtual machine (a "Podman machine"). On Windows, that machine is a WSL distribution.

The container has no network access, so if a malicious document compromises it, it can't phone home. Inside the container, the conversion runs under gVisor, an application kernel written in Go that implements a substantial portion of the Linux system call interface. gVisor sits between the conversion tools and the real kernel: an exploit that takes over LibreOffice still has to break out of gVisor, then out of the container, before it reaches your system. We wrote about this design in a blog post.

The image is signed with Cosign and updated independently from Dangerzone, so that vulnerabilities in the tools inside it can be patched quickly. See Independent sandbox updates.

What's lost

Because the safe PDF is rebuilt from pixels, some information is lost in the process:

- Links, forms, bookmarks and embedded files are gone.
- Text is only searchable and selectable if you enable OCR, and unfortunately OCR is never perfect.
- Metadata is destroyed. For Dangerzone this is a feature: metadata is one of the two things it promises to remove. See Security model.

For a document you receive from someone you don't trust, these are usually good trades.

Security model

This page describes what Dangerzone promises, what it assumes, and how its defenses are layered. It is the reasoning behind the security policy, which states what we consider a reportable vulnerability.

Goals

Dangerzone has two main security goals:

- Malicious documents should not infect the user's device, or communicate with other machines.
- All metadata should be destroyed after the conversion process.

Any vulnerability that undermines these two goals is considered critical. Everything else about Dangerzone (its usability, the quality of the output, its performance) matters, but is not a security property.

Assumptions

Dangerzone uses several third-party tools to sanitize documents, such as LibreOffice and PyMuPDF. Because these tools have a large attack surface, Dangerzone operates under the assumption that a 0-day vulnerability probably exists for them. In other words, the design does not rely on the document parsers being correct. It expects that, sooner or later, a document will take control of the process that opens it. That's why the untrusted document is only ever opened in a secure sandbox, where taking control of the process gains the attacker nothing.

Dangerzone does not use antivirus software to ensure that a file can be opened, nor does it disarm the file by removing just the "unsafe" components. First, antivirus software relies on previously seeing the same exploit in the wild, which is usually not the case for 0-days. Second, content disarm software assumes that file formats have "safe" and "unsafe" parts, and that if you snip the latter, file viewers won't have a problem with parsing the former. Our experience is that file parsers can have implementation bugs even on relatively simple parts of a specification, and that it's possible to break the readability of a file this way, leading to users opening the original.

So where does this leave us? Dangerzone's main assumption is that, at least for viewing purposes, documents and images can be reduced to their simplest possible representation: an RGB pixel stream per page. Reconstructing a safe PDF from such a pixel stream should then be the safest way to view this file.

Defense in depth

Isolation. The parsers run inside an unprivileged, networkless container. Inside the container, they run under gVisor, an application kernel that reimplements the Linux system call interface in Go and forwards only a small, hardened set of calls to the real kernel. To reach the host, an attacker needs a working exploit for the parser, then a gVisor escape, then a container escape.

Read more in our blog. On Qubes OS, the container is replaced by an offline disposable qube, which is destroyed after the conversion.

A narrow channel. The only thing that leaves the sandbox is a stream of page sizes and raw RGB pixels, read by Dangerzone with strict bounds (a maximum number of pages and a maximum page size). There is no file sharing and no network. The host never parses the untrusted document, and never parses anything produced by the untrusted document beyond those integers and pixel buffers.

No network. The sandbox has no network access, so a compromised parser cannot exfiltrate the document or fetch a second stage.

Metadata destruction. Rebuilding the PDF from pixels drops the original metadata as a matter of course, thus the safe PDF has no metadata in it.

Keeping the sandbox patched. Isolation is the primary defense, but a sandbox full of known vulnerabilities makes an attacker's job easier. Our second line of defense is to make sure our container image is not affected by known vulnerabilities, i.e., CVEs. We have nightly security scans for Critical CVEs, and biweekly security scans for High CVEs. You can see the results of our security scans in https://cves.dangerzone.rocks. We aim for a 4 week update cadence of our container image, or earlier, if a security finding necessitates it. Since 0.10.0, the sandbox image updates independently from Dangerzone releases, so a fix reaches users within days. See Independent sandbox updates.

Signed everything. Installers and packages are signed with the release PGP key, platform code signing is applied on macOS and Windows, and the sandbox image is signed with Cosign and verified before use. See Signing keys.

What this model does not cover

- **A compromised host.** If your machine is already compromised, Dangerzone cannot help. Its job is to protect the host from documents.
- **Content.** The safe PDF faithfully shows what the document looked like. If the document is a phishing letter, the safe PDF is a phishing letter you can read without being exploited.
- **Availability.** A document that makes the parsers loop forever or crash results in a failed conversion, which Dangerzone reports. That is an annoyance. The host stays untouched.

Vulnerabilities inside the sandbox

Because of the assumption above, a CVE in a tool inside the sandbox is expected and does not, on its own, compromise the security of Dangerzone. However, in combination with another more serious vulnerability (a container escape), a malicious document may be able to breach the security of Dangerzone. This is why we still publish security advisories and updated sandbox images for such CVEs, and why users are advised to keep both Dangerzone and its sandbox up to date.

Audit

Dangerzone received its first security audit by Include Security in December 2023. The audit was generally favorable, as it didn't identify any high-risk findings, except for 3 low-risk and 7 informational findings.

Reporting

If you believe you have found a vulnerability, please do not report it through public GitHub issues. Follow the security policy for the private channels and the timelines we commit to.

Independent sandbox updates

Dangerzone has a mechanism to auto-update the secure sandbox that is used for document conversion, independently from the application itself. This page explains why, and how the mechanism works. The commands are in Update the sandbox image.

Why decouple the sandbox from the application?

The sandbox image contains the tools with the largest attack surface: LibreOffice, the PDF renderer, and their dependencies. These receive security fixes quite often, and we consider them to be the most insecure part of Dangerzone. Before independent sandbox updates, shipping sandbox fixes meant a full Dangerzone release, which could take multiple days, if not weeks.

The independent sandbox update mechanism allows us to ship security fixes without having to do a full-blown release, shortening the time between the release of a security patch and its use. This increases the security of the conversion process dramatically, making it harder for an attacker to rely on known and patched exploits in our stack.

How Dangerzone learns about a new image?

Sandbox images are built and published to a container registry (`ghcr.io/freedomofpress/dangerzone/v1`) by the dangerzone-image repository. When update checks are enabled, Dangerzone periodically asks the registry for the "log index" of the latest image and compares it with what it has. Then, a new image is downloaded after the user confirms (or automatically, if they asked not to be prompted again).

The image name is versioned (`v1`) so that a change in the protocol between Dangerzone and the sandbox can be shipped as `v2` without breaking older applications.

Why you can trust a downloaded image?

Downloading executable code from the internet and running it, even inside a sandbox, deserves scrutiny. Three mechanisms make it safe:

Signatures. To make sure the sandbox image is trusted, we sign it with Cosign, which is part of the greater Sigstore ecosystem, and verify it against a key distributed in the Dangerzone application (`share/freedomofpress-dangerzone.pub`). That key is itself signed with our PGP release key. Dangerzone refuses to use an image whose signature cannot be verified.

Transparency log. Cosign signatures are recorded in Rekor, Sigstore's public transparency log. Dangerzone bundles and pins the Rekor public key, and keeps track of the log index of the image it has installed. An attacker who somehow manages to push an older, vulnerable container image as `latest` can't roll a user back to it: `load-archive` and the automatic updater refuse an image whose log index is lower than the installed one.

Build attestations. Each night, new images are built and pushed to the container registry, alongside a provenance attestation, so that anybody can verify that the image was built by

GitHub CI runners, from a defined source repository (`freedomofpress/dangerzone-image`). The images are reproducible, so anyone can rebuild them and compare.

Air-gapped machines

The sandbox with its signatures can travel on a USB stick, if required. The `dangerzone-image prepare-archive` command creates a tar file that can then be fed to `dangerzone-image load-archive`, which will verify the signatures and the log index on the air-gapped machine before installing the image.

What about the application itself

The application still updates through installers and package managers. Dangerzone only notifies you when a new release is out. The reasoning for that choice, and the alternatives we considered are listed in Update notifications.

Update notifications

This design document explains how the notification mechanism for Dangerzone updates works, what its benefits and limitations are, and what other alternatives we have considered. It has been adapted from discussions on GitHub issue #189, and has been updated to reflect the current design.

A user-facing document on how update notifications work can be found in Update Dangerzone. The keys mentioned below are documented in the settings reference, and the independent sandbox update mechanism that this design was later extended with is described in Independent sandbox updates.

Design overview

A hamburger icon is visible across almost all of the Dangerzone windows, and is used to notify the users when there are new releases.

First run

We detect it's the first time Dangerzone runs because the `settings["updater_last_check"] is None`.

Add the following keys in our `settings.json` file.

- `"updater_check_all": True`: Whether or not to check and apply independent container updates and check for new releases.
- `"updater_last_check": None`: The last time we checked for updates (in seconds from Unix epoch). `None` means that we haven't checked yet.
- `"updater_latest_version": "0.4.2"`: The latest version that the Dangerzone updater has detected. By default it's the current version.
- `"updater_latest_changelog": ""`: The latest changelog that the Dangerzone updater has detected. By default it's empty.
- `"updater_errors": 0`: The number of update check errors that we have encountered in a row.

Previously, `"updater_check"` was used to determine if we should check for new releases, and has been replaced by `"updater_check_all"` when adding support for independent container updates.

Second run

We detect it's the second time Dangerzone runs because `settings["updater_check_all"] is not None and settings["updater_last_check"] is None`.

Before starting up the main window, the user is prompted if they want to enable update checks.

Subsequent runs

We perform the following only if `settings["updater_check_all"] == True`.

1. Spawn a new thread so that we don't block the main window.
2. Check if we have cached information about a release (version and changelog). If yes, return those immediately.
3. Check if the last time we checked for new releases was less than 12 hours ago. In that case, skip this update check so that we don't leak telemetry stats to GitHub.
4. Hit the GitHub releases API and get the latest release. Store the current time as the last check time, even if the call fails.
5. Check if the latest release matches `settings["updater_latest_version"]`. If yes, return an empty update report.
6. If a new update has been detected, return the version number and the changelog.
7. Add a green bubble in the notification icon, and a menu entry called "New version available".
8. Users who click on this entry will see a dialog with more info:
    - Title: "Dangerzone v0.5.0 has been released"
    - Body:

      > A new Dangerzone version has been released. Please visit our downloads page to install this update.
      >
      > (Show unrendered changelog in Markdown format in a collapsible text box)

    - Buttons:
        - OK: Return

Notes:

- Any successful attempt to fetch info from GitHub will result in clearing the `settings["updater_errors"]` key.

Error handling

We trigger error handling when the updater thread encounters an error (either due to an HTTPS failure or a Python exception) and does not complete successfully.

1. Bump the number of errors we've encountered in a row (`settings["updater_errors"] += 1`)
2. Return an update report with the error we've encountered.
3. Update the hamburger menu with a red notification bubble, and add a menu entry called "Update error".
4. If a user clicks on this menu entry, show a dialog window:
    - Title: "Update check error"
    - Body:

> Something went wrong while checking for Dangerzone updates:
>
> You are strongly advised to visit our downloads page and check for new updates manually, or consult this page for common causes of errors. Alternatively, you can uncheck "Check for updates", if you are in an air-gapped environment and have another way of learning about updates.
>
> (Show the latest error message in a scrollable, copyable text box)

- Buttons:
  - Close: Return

Key Benefits

1. The above approach future-proofs Dangerzone against API changes or bugs in the update check process, by asking users to manually visit https://dangerzone.rocks.
2. If we want users to install a release immediately, we can say so in the release body, which we will show in a pop-up window.
3. If we are aware of issues that prevent updates, we can add them in the wiki page that we show in the error popup. Wiki pages are not versioned, so we can add useful info even after a release.

Security Considerations

Because this approach does not download binaries / auto-updates, it **does not add any more security issues** than the existing, manual way of installing updates. These issues have to do with a compromised/malicious GitHub service, and are the following:

1. GitHub pages can alter the contents of our main site (https://dangerzone.rocks)
2. GitHub releases can serve an older, vulnerable version of Dangerzone, instead of a new update.
3. GitHub releases can serve a malicious binary (requires a joint operation from a malicious CA as well, for extra legitimacy).
4. GitHub releases can silently drop updates.
5. GitHub releases can know which users download Dangerzone updates.
6. Network attackers can know that a user has Dangerzone installed (because we ask the user to visit https://dangerzone.rocks)

A good update framework would probably defend against 1,2,3. This is not to say that our users are currently unprotected, since 1-4 can be detected by the general public and the developers (unless GitHub specifically targets an individual, but that's another story).

Usability Considerations

1. We do not have an update story for users that only use the Dangerzone CLI. A good assumption is that they are on Linux, so they auto-update.

Alternatives

We did some research on this subject and found that there are update frameworks that do this job for us. While working on this issue, we decided that integrating with one framework will certainly take a bit of work, especially given that we target both Windows and MacOS systems. In the meantime though, we didn't want to ship releases without at least a notification channel, since staying behind on updates has a huge negative impact on the users' safety.

The update frameworks that we learned about are:

Sparkle Project

Sparkle project seems to be the de-facto update framework in MacOS. Integrators in practice need to care about two things: creating a proper `Appcast.xml` file on the server-side, and calling the Sparkle code from the client-side. These are covered in the project's documentation.

The client-side part is not very straight-forward, since Sparkle is written in Objective-C. Thankfully, there are others who have ventured into this before: https://fman.io/blog/codesigning-and-automatic-updates-for-pyqt-apps/

The server-side part is also not very straight-forward. For integrators that use GitHub releases (like us), this issue may be of help: https://github.com/sparkle-project/Sparkle/issues/648

The Windows platform is not covered by Sparkle itself, but there are other projects, such as WinSparkle, that follow a similar approach. I see that there's a Python library (`pywinsparkle`) for interacting with WinSparkle, so this may alleviate some pains.

Note that the Sparkle project is not a silver bullet. Development missteps can happen, and users can be left without updates. Here's an example issue that showcases this.

The Update Framework

The Update Framework is a graduated CNCF project hosted by Linux Foundation. It's based on the Thandy updater for Tor. It's not widely adopted, but some of its adopters are high-profile, and it has passed security audits.

It's more of a specification and less of a software project, although a well-maintained reference implementation in Python exists. Also, a Python project (`tufup`) that builds upon this implementation makes it even easier to generate the required keys and files.

Regardless of whether we use it, knowing about the threat vectors that it's protecting against is very important.

Other Projects

- Qt has some updater framework as well: https://doc.qt.io/qtinstallerframework/ifw-updates.html
- Google Chrome has its own updater framework: https://chromium.googlesource.com/chromium/src.git/+/master/docs/updater/protocol_3_1.md
- Keepass rolls its own update mechanism: https://github.com/keepassxreboot/keepassxc/blob/develop/src/updatecheck/UpdateChecker.cpp

- PyUpdater was another popular updater project for Python, but is now archived.

Reproducible builds

We want to improve the transparency and auditability of our build artifacts, and a way to achieve this is via reproducible builds. For a broader understanding of what reproducible builds entail, check out https://reproducible-builds.org/.

Our build artifacts consist of:

- Container images (`amd64` and `arm64` architectures)
- macOS installers (for Intel and Apple Silicon CPUs)
- Windows installer
- Fedora packages (for regular Fedora distros and Qubes)
- Debian packages (for Debian and Ubuntu)

As of writing this, the following artifacts are reproducible:

- Container images (see #1047). You can find detailed documentation on reproducible containers in the dangerzone-image repository.
- Debian packages (`dangerzone` and `dangerzone-full`).

Debian packages

Building the `.deb` files inside our pinned Debian Bookworm dev container (`./dev_scripts/env.py --distro debian --version bookworm`) and via `./install/linux/build-deb.py` produces byte-identical packages on every run, given the same source tree and the same `share/container.tar`.

The mechanisms that make this work:

- **`SOURCE_DATE_EPOCH`** is pinned to the timestamp of the latest `debian/changelog` entry. `dpkg-source`, `dpkg-deb`, and `dh_builddeb` use this value for file mtimes inside the source tarball and the binary `.deb`, and for the `Date` field in `.changes` / `.dsc` files. The value is set by `debian/rules` (so it's correct even when invoking `dpkg-buildpackage` directly) and re-asserted by `install/linux/build-deb.py`.
- **Locale and timezone** are forced to `C.UTF-8` / `UTC`, so any tool that emits sorted output or formatted timestamps does so identically across machines.
- **File mtimes are clamped** to `SOURCE_DATE_EPOCH` after `dh_install` populates the staging trees. `install` and pybuild do not preserve timestamps, so without this step the embedded `data.tar` would carry the wall-clock time of the build.
- **The build environment is pinned** via the Debian Bookworm dev container, which fixes the versions of `dpkg`, `debhelper`, `pybuild`, and the Python interpreter that participate in the build.

Verifying reproducibility

To confirm a `.deb` is reproducible, build it twice in clean environments and compare the outputs:

```
# First build
./dev_scripts/env.py --distro debian --version bookworm run --dev bash -c \
```

```
    "cd dangerzone && ./install/linux/build-deb.py"
mv deb_dist deb_dist.first

# Clean and build again
git clean -fdx debian/ deb_dist/
./dev_scripts/env.py --distro debian --version bookworm run --dev bash -c \
    "cd dangerzone && ./install/linux/build-deb.py"

# Compare
sha256sum deb_dist.first/*.deb deb_dist/*.deb
# For a deeper diff if hashes differ:
#   diffoscope deb_dist.first/dangerzone_<ver>_amd64.deb deb_dist/dangerzone_<ver>_amd64.deb
```

The two sets of `.deb` files should have matching SHA-256 sums. If they don't, `diffoscope` will show what diverged. Common culprits are unpinned timestamps, locale-dependent sort order, or files generated outside the staging trees.

Qubes OS integration

On Qubes OS, Dangerzone can run without containers: the conversion happens in a disposable qube, which is what Qubes OS is built for. This page explains how that integration is put together. The installation steps are in Installation, the development setup in Development environment. The integration is in beta.

The conversion runs in a disposable qube created from an offline template (`dz-dvm`), through Qubes RPC. Here, no container images are used at all, and update checks are skipped.

It is also possible to select the Qubes converter by setting the `QUBES_CONVERSION=1` environment variable.

What happens during a conversion

1. The application calls `qrexec-client-vm @dispvm:dz-dvm dz.Convert`. Qubes OS starts a fresh disposable qube from `dz-dvm`, provided the RPC policy in `/etc/qubes/policy.d/50-dangerzone.policy` allows it.
2. The untrusted document is written to the RPC call's standard input. The `dz.Convert` service in the disposable qube runs the same document-to-pixels code that the container image runs, and writes the page count, page sizes, and RGB pixels to standard output. See the sandbox protocol for more details on this.
3. The application rebuilds the PDF from the pixels in the app qube, exactly as on other platforms.
4. The disposable qube is destroyed when the call ends.

Because `dz-dvm` has no network (`netvm=""`) and cannot spawn further disposables (`default_dispvm=''`), a compromised converter is stuck in a qube that disappears. The second setting is the one the 2023-10-25 advisory added.

What the packages contain

The `dangerzone-qubes` RPM drives the conversion from the host's side. It is built from the `freedomofpress/dangerzone` repo with `./install/linux/build-rpm.py --qubes` and has slightly different dependencies than the regular Dangerzone package. Most importantly, it depends on `dangerzone-insecure-converter-qubes`, which drives the conversion from within the disposable qubes. It is built from the `freedomofpress/dangerzone-image` repo with `./qubes/build-rpm.sh`. It installs an RPC service for Dangerzone under `/etc/qubes-rpc` and dependencies like LibreOffice, so that the template has everything the disposable qube needs.

Development: `dz.ConvertDev`

Rebuilding and installing the RPM in the template for every change to the converter is not very practical. In development (`DANGERZONE_DEV=1`), Dangerzone calls `dz.ConvertDev` instead, and first sends a zip of the converter's Python sources (from `DANGERZONE_INSECURE_CONVERTER_PATH`) over the RPC channel. The disposable qube unpacks and runs that code, so changes to the

converter are picked up immediately. The RPC policy for development allows both `dz.Convert` and `dz.ConvertDev`.

Limitations

- Hancom HWP files are not supported on Qubes OS.
- Support is manual QA only on the latest Fedora template, see Operating System support.
- The user must manually enable the respective Dangerzone policy in dom0, as part of the installation instructions.

Create Dangerzone environments

The `dev_scripts/env.py` script creates environments where a user can run Dangerzone, allows the user to run arbitrary commands in these environments, as well as run Dangerzone (nested containerization). Because Dangerzone supports a large number of Linux distributions, this is a helper to test Dangerzone in these various setups.

It supports two types of environments:

1. Dev environment. This environment has developer tools, necessary for Dangerzone, baked in. In this case, it mounts the Dangerzone source under `/home/user/dangerzone` in the container. The developer can then run Dangerzone from source, with `poetry run dangerzone`.

2. End-user environment. This environment has only Dangerzone installed in it from one of the following sources:

   - *(default)*: from the .deb/.rpm package that we have created
   - `--qa`: from https://packages-qa.freedom.press
   - `--prod`: from https://packages.freedom.press

   For convenience, it also has the Dangerzone source mounted under `/home/user/dangerzone`, but it lacks Poetry and other build tools. The developer can run Dangerzone there with `dangerzone`. This environment is the most vanilla Dangerzone environment, and should be closer to the end user's environment, than the development environment.

Each environment corresponds to a Dockerfile, which is generated on the fly. The developer can see this Dockerfile by passing `--show-dockerfile`.

For usage information, run `./dev_scripts/env.py --help`.

Nested containerization

As Dangerzone environments are containers, the isolation sandbox that Dangerzone creates is a nested container. This brings the following challenges:

1. Containers typically only have a subset of syscalls allowed, and sometimes only for specific arguments. This happens with the use of seccomp filters. For instance, in Docker, the `clone` syscall is limited in containers and cannot create new namespaces (https://docs.docker.com/engine/security/seccomp/#significant-syscalls-blocked-by-the-default-profile). For testing/development purposes, a seccomp filter for the external container is used to get around this limitation, with `--security-opt seccomp=unconfined`. This has the same effect as developing Dangerzone locally, so it should probably be sufficient for now.

2. While Linux supports nested namespaces, extra handling is required for nested user namespaces. By default, the configuration for each user namespace (see `man login.defs`) is to reserve 65536 UIDs/GIDs, starting from UID/GID 100000. This works fine for the first container, but can't work for the nested container, since it doesn't have enough UIDs/

GIDs to refer to UID 100000. Our solution to this is to restrict the number of UIDs/GIDs allowed in the nested container to 2000, which should be enough to run `podman` in it.

3. Containers also restrict the capabilities (see `man capabilities`) of the processes that run in them. By default, containers do not have mount capabilities, since it requires `CAP_SYS_ADMIN`, which effectively makes the process root in the specific user namespace. In our case, we have to give the Dangerzone environment this capability, since it will have to mount directories in Podman containers. For this reason, as well as some extra things we bumped into during development, we pass `--privileged` when creating the Dangerzone environment, which includes the `CAP_SYS_ADMIN` capability.

GUI containerization

Running a GUI app in a container is a tricky subject for multi-platform apps. In our case, we deal specifically with Linux environments, so we can target just this platform.

To understand how a GUI app can draw in the user's screen from within a container, we must first understand how it does so outside the container. In Unix-like systems, GUI apps act like clients to a display server. The most common display server implementation is X11, and the runner-up is Wayland. Both of these display servers share some common traits, mainly that they use Unix domain sockets as a way of letting clients communicate with them.

So, this gives us the answer on how one can run a containerized GUI app; they can simply mount the Unix Domain Socket in the container. In practice this is more nuanced, for two reasons:

1. Wayland support is not that mature on Linux, so we need to set some extra environment variables. To simplify things, we will target X11 / XWayland hosts, which are the majority of the Linux OSes out there.
2. Sharing the Unix Domain socket does not allow the client to talk to the display server, for security reasons. In order to allow the client, we need to mount a magic cookie stored in a file pointed at by the `$XAUTHORITY` envvar. Else, we can use `xhost`, which is considered slightly more dangerous for multi-user environments.

Caching and Reproducibility

In order to build Dangerzone environments, the script uses the following inputs:

- Dev environment:
  - Distro name and version. Together, these comprise the base container image.
  - `poetry.lock` and `pyproject.toml`. Together, these comprise the build context.
- End-user environment:
  - Distro name and version. Together, these comprise the base container image.
  - `.deb` / `.rpm` Dangerzone package, as found under `deb_dist/` or `dist/` respectively.

Any change in these inputs busts the cache for the corresponding image. In theory, this means that the Dangerzone environment for each commit can be built reproducibly. In practice, there are some issues that we haven't covered yet:

1. The output images are:
    - Dev: `dangerzone.rocks/build/{distro_name}:{distro_version}`
    - End-user: `dangerzone.rocks/{distro_name}:{distro_version}`

    These images do not contain the commit/version of the Dangerzone source they got created from, so each one overrides the other.
2. The end-user environment expects a `.deb` / `.rpm` tagged with the version of Dangerzone, but it doesn't insist on being built from the current Dangerzone commit. This means that stale packages may be installed in the end-user environment.
3. The base images may be different in various environments, depending on when they were pulled.

State

The main goal behind these Dangerzone environments is to make them immutable, so that they do not need to be stored somewhere, but can be recreated from their images. Any change to these environments should therefore be reflected in their Dockerfile.

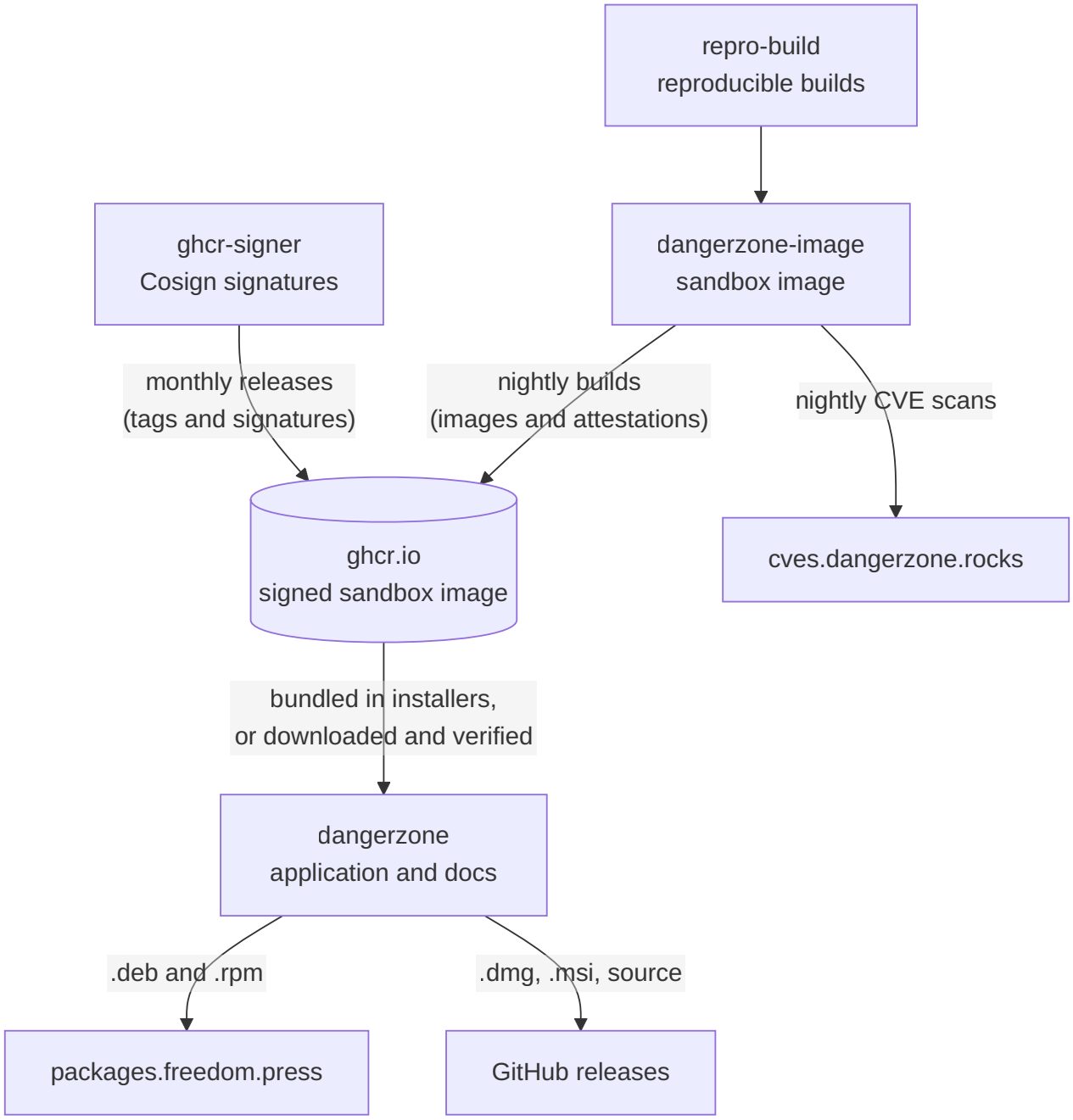
To enforce immutability, we delete the containers every time we run a command or an interactive shell exits. This means that these environments are suitable only for running Dangerzone commands, and not doing actual development in them (install an editor, configure bash prompts, etc.)

The only point where we allow mutability is the directory where Podman stores the images and stopped containers, which may be useful for developers. If this proves to be an issue, we will reconsider.

The Dangerzone repositories

Dangerzone is more than one repository. The application, the sandbox image it runs, the packages users install and the signatures that tie them together, each live in their own place. This page gives contributors the bird's-eye view: what each repository or site is for, and how they fit together.

The big picture



Every box in the graph is a link to the repository or site it stands for.

The repositories

freedomofpress/dangerzone

The main repository: the desktop application (in Qt), the CLI tools, the packaging for every platform and this documentation site. Releases are tagged here and their installers are published on the GitHub releases page.

freedomofpress/dangerzone-image

The sandbox: the container image that performs the document-to-pixels conversion, and the Python package that runs inside it. Images built from the `main` branch are published nightly to `ghcr.io/freedomofpress/dangerzone/v1` (but are not tagged as `latest`), whereas images from feature branches are published under a testing namespace, `ghcr.io/freedomofpress/dangerzone-testing/v1`. The CI of this repo builds the image reproducibly, attaches provenance attestations, runs the daily CVE scans, and publishes the security dashboard. Problems with a document format, a bundled tool, or a CVE inside the sandbox belong here. The interface between the application and the image is described in Sandbox protocol, and the update mechanism in Independent sandbox updates.

freedomofpress/packages

Repository for the `.deb` and `.rpm` packages, served on https://packages.freedom.press (and on https://packages-qa.freedom.press for release candidates).

freedomofpress/ghcr-signer

Utilities to publish our official container images on a monthly cadence. Candidate images from our nightly builds are signed privately with a hardware key and the signatures are submitted as a pull request. The CI verifies the signatures and once the pull request is merged, it pushes them to the GitHub Container Registry and tags the image as `latest`. The blog post The GHCR Signer explains why this is needed and how it works. This is the step that lets Dangerzone trust a downloaded sandbox image.

freedomofpress/repro-build

A helper script to build bit-for-bit reproducible container images, by providing a build environment that is consistent across operating systems and container engines. `dangerzone-image` uses it so that anyone can rebuild the sandbox image and compare digests. The blog post Reproducing the reproducible images tells the story, and Reproducible builds covers what is reproducible on the Dangerzone side.

freedomofpress/dangerzone-rs

A rewrite of dangerzone in rust, currently in the works and not to be used yet.

Websites

stats.dangerzone.rocks

Release statistics for the project aggregating data from the GitHub releases and the container registry. Generated by freedomofpress/dangerzone-stats.

cves.dangerzone.rocks

The security dashboard: the results of the nightly grype scans of the sandbox image, both for the development branch and the latest released image. This is the second line of defense described in the security model. The dashboard is published from the `dangerzone-image` repository.

dangerzone.rocks and this site

The official website, with download links and the blog. Built from freedomofpress/dangerzone.rocks. This documentation site is built from the `docs/` directory of the main repository and published at docs.dangerzone.rocks.

Frequently asked questions

What is a "safe PDF"?

A PDF that Dangerzone rebuilt from pictures of each page of the original. It contains only those pictures and, if you enabled OCR, a text layer. That removes any script, embedded files or logic that was present in the original document. See How Dangerzone works.

Can Dangerzone tell me whether a file is malicious?

No. Unfortunately, Dangerzone can't tell you if a file is safe. It just creates a copy that is.

What is OCR, and why does Dangerzone ask for a language?

OCR stands for "Optical Character Recognition": software that reads the text in a picture. Dangerzone uses it to add an invisible text layer, which makes the documents searchable and copy-pastable. OCR works much better when it knows which language to expect, hence the language choice.

Does Dangerzone remove watermarks and tracking information?

It depends on where the information lives.

- **Metadata** (author, software, file creation dates, camera model, GPS coordinates, editing history) is removed as part of the process.
- **Hidden document structure** (comments, embedded files, scripts, links) is also removed.
- **Anything visible on the page** will not be removed. For example, visible watermarks, printer tracking dots, unusual spacing, unique wording, or steganography hidden in the image will not be removed. If you are a source, treat this as a separate risk that Dangerzone does not address. See When should I use Dangerzone?.

Is there a command-line version?

Yes. `dangerzone-cli` is installed next to the graphical application on every platform, and can convert files from the terminal. See Convert documents from the command line and the dangerzone-cli reference.

Why not just open PDFs in Firefox, which has its own sandbox?

Browser viewers such as `PDF.js` render the PDF inside the browser's sandbox, which does contain many attacks. Two differences remain. First, the PDF is still the original file: once you save or forward it, the next person gets every script and link it carried. Second, browser

sandboxes are a target in their own right and escapes are found regularly. Dangerzone gives you a new file that is safe in any viewer, produced in a sandbox that runs nothing else and has no network.

Isn't this overkill for everyday use?

For files you trust, yes. Dangerzone is aimed at people who regularly open files from strangers, such as journalists, lawyers and researchers (or anyone handling one specific suspicious file, really). Converting takes seconds for a short PDF and the result is heavier and less editable than the original. Use it when you feel a file is suspicious, and see When should I use Dangerzone? for the cases where it isn't the right tool.

Why would I convert a JPG or PNG? They are just pictures.

Image formats are decoded by complex libraries, and those libraries have had serious vulnerabilities in the past. Exploits delivered through crafted images, including PNG and SVG files, have been used in the wild. Dangerzone decodes the image inside the sandbox and re-encodes only the pixels. It also strips image metadata, such as camera model and GPS coordinates, which is often what people want when sharing a picture.

Why is the safe PDF so large?

Every page is stored as an image. Dangerzone compresses the result, but a picture of a page of text is still larger than the text itself. Long documents with OCR take the longest and produce the biggest files.

Has Dangerzone received a security audit?

Yes, Dangerzone received its first security audit by Include Security in December 2023. The audit was generally favorable: it didn't identify any high-risk findings, only 3 low-risk and 7 informational ones.

Can I run Dangerzone in an airgapped environment?

Yes, Dangerzone is designed to run in airgapped environments without any configuration. If you want to update its container image, follow our instructions.

Can I use a custom runtime, such as Podman Desktop?

On Windows and macOS, Dangerzone embeds Podman, so there is no need to.

To use a different podman version, such as Podman Desktop, follow our documentation.

I'm experiencing an issue while using Dangerzone

First, make sure you use the latest version of Dangerzone.

If the problem persists, please report it.

Why does Dangerzone need to download something on first start?

The sandbox is a container image of a few hundred megabytes. On macOS and Windows it is bundled in the installer. On Linux, the slim `dangerzone` package downloads it on first use so that the package itself stays small and the sandbox can be updated independently. The `dangerzone-full` package bundles it instead. See Independent sandbox updates.

Which platforms and document formats are supported?

See Supported platforms and Supported document formats.

Where can I report security issues?

Please check our security policy for preferred communication methods, and for what's in scope for security issues.

How can I become involved?

We welcome contributors to the project, regardless of their level of expertise. We don't have a place where we hang out at the moment, but you can reach out to us either via a GitHub issue or `support <at> dangerzone.rocks`!

If you are a programmer, feel free to pick up an issue labelled good first issue and set up a development environment.